Development of a RISC-V Processor with Integrated AES Hardware Acceleration

Writers: Gilad Navok and Ofek Baribi

Supervisor: Etay Sela

Lab: VLSI

Table of Contents

Abstract

The design is a low-power RISC-V processor for IoT devices equipped with an embedded AES accelerator. It implements the unprivileged `RV32I_Zicsr` ISA with M-mode CSRs and interrupts and extends it with custom cryptographic instructions. The micro-architecture is in-order, three-stage pipeline with non-uniform stage lengths, optimized around a lightweight memory interface that delivers instructions in two cycles.

Introduction

The RISC-V ISA

The RISC-V ISA is a Reduced Instruction Set Computing architecture defined by its open, royalty-free specifications. Unlike proprietary ISAs, it is managed by RISC-V International, a non-profit association the fosters collaboration between industry and academia. The architecture is defined by its modular design, featuring a small, frozen base instruction set that can augmented with optional extensions. This structure makes RISC-V highly modifiable and extendable, allowing designers to tailer the processor to specific needs. While it is still emerging in the general consumer processor market, its flexibility has made it a dominant choice for micro-controllers, IoT and edge computing devices.

Hardware Acceleration

As the performance scaling predicted by Moore's Law slows down, the industry has entered an "acceleration era" where computationally intensive workloads are offloaded to dedicated hardware. Hardware accelerators can offer orders-of-magnitude speed-ups compared to general-purpose CPUS, but this advantage is highly dependent on targeting the right workloads. Since additional silicon area and engineering efforts are expensive resources, any dedicated hardware must provide sufficient performance improvements to justify its cost. Common examples of this trade-off include GPUs for parallel floating-point operations and Google TPUs for matrix multiplication, both of which sacrifice general flexibility for extreme domain-specific efficiency.

Processor Architecture

Architectural Overview

The core implements the following features:

- **ISA:** Unprivileged `RV32I_Zicsr` with M-mode CSRs and interrupts.
- **Extensions:** Custom cryptographic instructions for hardware-accelerated encryption and decryption.
- **System:** Support for system and external interrupts.

The pipeline contains three stages, with their main roles being:

- Instruction Fetch (IF): Interfaces with IMEM to fetch instructions, manages the PC and buffers instructions.
- Instruction Decode (ID): Translates instructions to control signals and initiates execution.
- Execution (EXE): Performs arithmetic, comparisons, cryptographic operations, writes back results and interfaces with DMEM.

To optimize around the low-power, 2-cycle memory interface, a significant part of the core uses 16 bit data paths and 16 bit hardware. Instructions are executed over three execution cycles:

1. Cycle 1: Takes place in the ID stage.
2. Cycle 2: Takes place in both the ID and EXE stages.

3. Cycle 3: Takes place in the EXE stage.

Pipeline Flow

The execution of each instruction is split into two *steps* per stage. We designate these execution steps as ID1, ID2, EXE1 and EXE2. ID1 takes place the first execution cycle, ID2 and EXE1 both take place in the second execution cycle and EXE2 takes place in the third execution cycle. Each instruction advances through five time-slots:

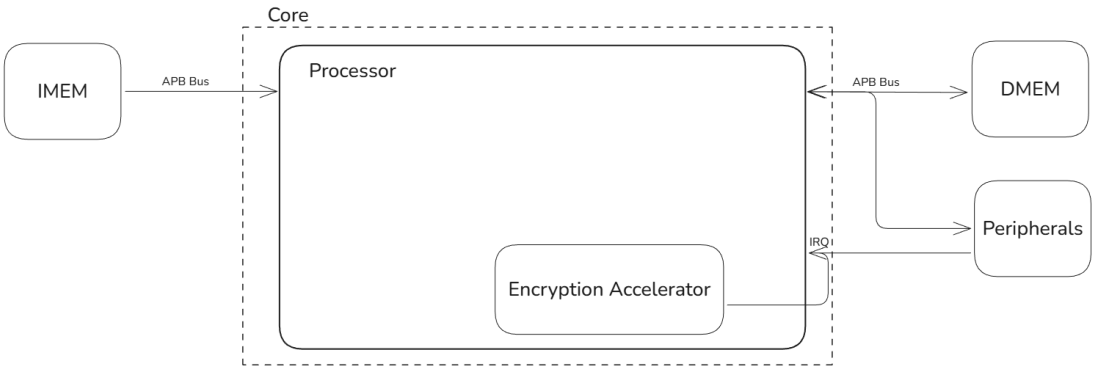
1. IF 0 - IF 1 : Two cycles to fetch the instruction over the 2-cycle memory interface.
2. ID1 (First Execution Cycle) : Decode the instruction, and perform the first ID execution step.
For example:
    - *Arithmetic:* Read `rs2` and buffer its lower half.
    - *Load:* Calculate load address and start DMEM read transfer.
3. ID2/EXE1 (Second execution cycle) : Perform the second ID execution step and the first EXE execution step.
For example:
    - *Arithmetic:* ID forwards the lower half of `rs1` and buffers the upper halves of `rs1`, `rs2`. EXE performs an arithmetic operation on lower halves and writes back the result to the lower half of `rd`.
    - *Load:* ID holds the address. EXE stalls until the transfer finishes and writes back the lower half.
4. EXE2 (Third execution cycle) : Perform the second EXE execution step.
    - *Arithmetic:* Same as first step but on the upper halves.
    - *Load:* Writes back the upper half of the transferred data.

Instruction Timeline

Instruction/Cycle	1	2	3	4	5	6	7	8	9
I1	IF	IF	ID	ID/EX	EX				
I2			IF	IF	ID	ID/EX	EX		
I3					IF	IF	ID	ID/EX	EX

Stage/Cycle	1	2	3	4	5	6	7	8	9
IF	i1	i1	i2	i2	i3	i3	i4	i4	i5
ID	i0	i0	i1	i1	i2	i2	i3	i3	i4
EXE	i(-1)	i0	i0	i1	i1	i2	i2	i3	i3

Interfaces



The system is designed around a modified Harvard architecture, utilizing distinct interfaces for instruction and data access. The Core encapsulates the main RISC-V Processor and the tightly coupled Encryption Accelerator. It interacts with the external environment through three primary channels:

- **Instruction Memory (IMEM) :** A dedicated read-only APB interface used by the Fetch Unit to retrieve program code.
- **Data Memory (DMEM):** A read/write APB interface that handles load/store operations to data memory and memory-mapped peripherals.
- **Interrupt Requests (IRQ):** A dedicated line allowing external peripherals to trigger asynchronous interrupts in the processor.

Interrupt Interface and Rules

The processor manages asynchronous events through a centralized interrupt controller (`interrupt_sbm`), which handles both external system interrupts and internal accelerator notifications.

External Interrupt Interface (`interrupt_req_ext_i`):

- **Description:** A single, synchronous input pin at the top level of the core.
- **Mapping:** Asserting this pin sets bit 11 (Machine External Interrupt) in the `mip` (Machine Interrupt Pending) CSR. If unmasked, it triggers a trap with `mcause` (Machine Cause Register) = 11.
- **Interface Rules:** This is a **level-sensitive** input. The external system (such as a PLIC or a dedicated peripheral) must assert and hold this line HIGH. The signal must remain stable until the processor's software trap handler explicitly clears the source of the interrupt via a memory-mapped I/O transfer over the DMEM APB bus.

Internal Accelerator Interrupt:

- **Description:** A dedicated internal routing channel from the AES-128 accelerator to the interrupt controller.
- **Mapping:** When the accelerator finishes an operation, it asserts an internal signal that sets bit 16 in the `mip` CSR. If unmasked, this triggers a trap with `mcause` = 16.
- **Interface Rules:** Because this is contained entirely within the core wrapper, it requires no external pin wiring or top-level system integration.

Configurable System Parameters

To allow system integrators to optimize the core for specific area, power, and performance targets, the core exposes several key hardware parameters at the top level of the design.

Core & Memory Parameters

Parameter	Default Value	Description
PREFETCH_BUFFER_CAPACITY	2	**Fetch Depth:** Determines the number of instructions the IF stage can buffer. Tuning this

Parameter	Default Value	Description
		allows integrators to balance instruction throughput against hardware register area.
INIT_PC	0	**Reset Vector:** The memory address the Program Counter initializes to upon reset (`rst_n`).

AES Accelerator Parameters (Area vs. Latency Trade-off)

The embedded AES accelerator allows the user to configure the number of parallel Substitution Boxes (S-Boxes) instantiated in hardware. This parametric design enables the integrator to perfectly balance the encryption latency against the available silicon area.

Parameter	Default Value	Description
SBOX_PAR_ROUND	16	Number of parallel S-Boxes used during the standard encryption round.
SBOX_PAR_INV_ROUND	16	Number of parallel S-Boxes used during the inverse decryption round.
SBOX_PAR_KEY	4	Number of parallel S-Boxes allocated for the Key Expansion schedule.

Hazards and Resolution

One significant advantage of the architecture's 16-bit split datapath is the natural mitigation of many Read-After-Write hazards. Since most operations access only one half of a register (lower or upper) at a time, instruction can often operate on the same register simultaneously without conflict, provided they target different halves.

However, specific pipeline sequences still create data or structural dependencies. The system handles four specific hazard types:

- **Store-Load Hazard (Structural)**
  - **Description:** Occurs when a Store instruction in the EXE stage is followed immediately by a Load instruction in the Decode stage that attempts to use the "Load Bypass" mechanism. The Load Store Unit cannot initiate a new APB transfer for the Load while it is still finalizing the Store transfer.
  - **Solution: Stall**. The Decode stage holds the Load instruction until the Store completes and the APB interface is ready.
- Full Read-After-Write on RS1
  - **Description:** Occurs when the instruction in Decode requires the value of `rs1` immediately during its first cycle (e.g., for Jump targets, Branch comparisons, or Load/Store address calculations), but the instruction in EXE stage is currently writing to the same register.
  - **Solution: Stall.** The instruction waits one cycle for the write to complete so the correct value can be read from the register file.
- **Half Read-After-Write on RS2**
  - **Description:** A subtle hazard where the instruction in EXE stage is writing to one half of a register, and the instruction in ID stage attempts to read that exact same half from `rs2` for an ALU operation. This usually happens due to a mismatch between the write-back order (flipped vs. normal) and the read serialization start (upper vs. lower).

    - **Solution: Stall.**
    - **Rationale**: While forwarding is possible, it would require injecting a Mux into the main data path before the Serializer. Since the Serializer input already selects between four sources, adding another layer of logic here extends the critical path. A stall was chosen to keep the main datapath clean and fast, as this hazard is architecturally rare.
- **Full Read-After-Write on RS2**
    - **Description:** Occurs when the instruction in ID stage requires the full 32-bit value of `rs2` (typically for a Store operation), and the instruction in EXE stage is currently writing to it.
    - **Solution: Forwarding (Internal Patching).**
    - **Rationale:** Unlike the Half-RAW case, the data here is required for the subsequent cycle. Since the Register File read port is often allocated to other operations during that cycle (such as reading `rs1`), the data cannot simply be re-read. Therefore, the forwarding logic targets an internal holding register rather than the immediate output of the serializer. This removes the serializer logic from the forwarding timing path, allowing the data to be captured safely without affecting the processor's maximum frequency.

The AES Accelerator Subsystem

Accelerator Integration

Interface Definition

- **Data Interface (Registers x28-x31)**: The accelerator is tightly coupled to the core via the upper General Purpose Registers.
    - **Design Rationale:*** Utilizing existing Register File entries eliminates the need for dedicated memory-mapped I/O buffers or complex bus protocols. This approach reduces hardware area and allows the software to prepare data using standard RISC-V base instructions.
    - **Dual Access:** These registers serve as the shared memory space; the core writes input plaintext/key data here, and reads the resulting ciphertext from the same location.
- **Control Interface:** Beyond data transfer, the interface includes specific control signals driven by the core's instruction decoder. These signals govern the accelerator's state machine:
    - `start_enc` / `start_dec`: Signals to initiate the encryption of decryption processes.
    - `load_key`: Signals the accelerator to latch the current register values as the new encryption key.

Parallel Execution Model

To enable high-performance operation, the system supports concurrent execution between the core and the accelerator. This is achieved through a "swap-on-start" mechanism:

1. Load: The core writes the input block into registers `x28-x31`
2. Signal & Swap: The core issues the `start` instruction. In this single cycle, the accelerator latches the input values from the registers and simultaneously writes the results of the previous operation back into them.
3. Parallel Processing: Once the signal is asserted, the accelerator begins its work. The registers `x28-x31` are immediately released, allowing the core to read the valid results and load the next data block while the accelerator is running in the background.
4. Pipeline Loop: This cycle repeats, effectively pipelining the data movement with the cryptographic computation.

Timing and Synchronization

- Latency Balancing: The accelerator design allows for a parametric number of encryption cycles. This parameter can be tuned so that the encryption duration roughly matches the core's software overhead (loading/storing data and loop management). When balanced, this hides the accelerator's latency completely behind the core's memory operations.

- Synchronizations Modes: To accommodate different software needs, the integration supports two synchronization methods:
    - Polling/Wait Mode: The core explicitly checks for status or halts until the accelerator finishes.
    - Interrupt Mode: The accelerator notifies the core upon completion.

This dual-mode synchronization allows the core to interface efficiently with both fixed, short-duration accelerators (using Sync/Stalls) and variable-latency or long-duration accelerators (using Interrupts).

System Control

Control and Status Registers

The core supports the RISC-V `Zicsr` extension providing instructions for atomic read and write operations to Control and Status Registers (CSRs).

The core implements the minimal set of Machine-mode CSRs required to support interrupt handling. Those registers are defined as follows:

Status and Control:

- `mstatus` (Machine Status): Contains global control and status information. Currently, the core implements the Machine Interrupt Enable (`mstatus.mie`) and Previous Machine Interrupt Enable (`mstatus.mpie`) bits to manage interrupt masking and nesting. Only bits 3 (`mie`) 7 (`mpie`) are writeable; bits 11, 12 are hardwired to 1 (indicating the core is in Machine Mode only setting); all other bits are hardwired to 0.
- `mie` (Machine Interrupt Enable): Masks specific interrupt sources. Each bit corresponds to a unique interrupt source (e.g., External or Accelerator), determining if that specific source is allowed to trigger a trap.
- `mip` (Machine Interrupt Pending): Indicates the status of pending interrupts. Each bit corresponds to a specific interrupt type, reflecting whether a request is currently active and awaiting service.

Trap Handling

- `mtvec` (Machine Trap-Vector Base-Address): Stores the base address of the trap handler. When an interrupt occurs, the PC is set to the value in this register. The core implements the direct mode trap-handling.
- `mepc` (Machine Exception Program Counter): When an interrupt occurs, the address of the interrupted instruction (or the next instruction to execute) is saved here. `mret` uses this value to resume execution.
- `mscratch` (Machine Scratch): A dedicated read/write register for the trap handler. It is typically used by the software to temporarily save a general-purpose register at the start of an interrupt service routine.

Trap Information

- `mcause` (Machine Cause): Indicates the architectural reason for the trap. The most significant bit denotes if the trap was an interrupt (1) or an exception (0), while the remaining bits form an exception code identifying the specific source.
- `mtval`(Machine Trap Value): Contains exception-specific information to assist software in handling the trap. For the currently supported interrupts this register is set to 0.
- `mtinst`(Machine Trap Instruction): Provides the instruction binary that caused the trap. Hardwired to zero in this implementation.

Bit Allocations

`mie, mip`:

- Bit 11: Machine External Interrupts.
- Bit 16: Accelerator Interrupts.

Interrupts

The interrupt controller (`interrupt_sbm`) accepts signals from two primary sources: the core's external interrupt interface and the internal embedded accelerator. External interrupts are prioritized over accelerator interrupts.

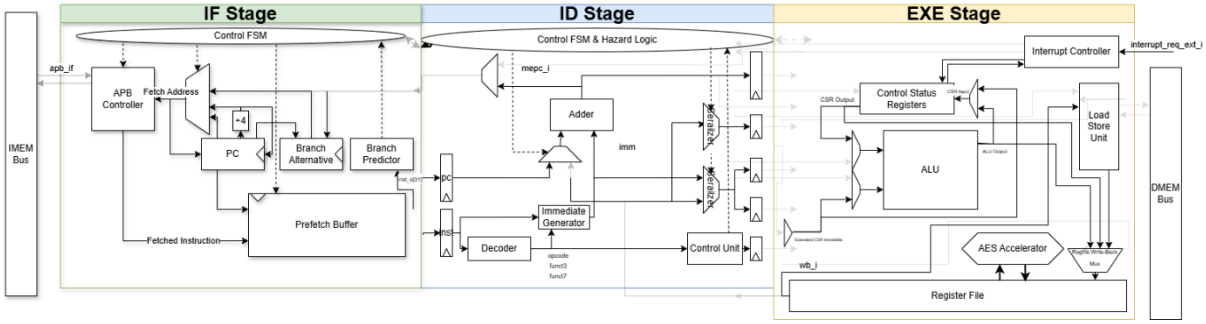
Interrupts in the core are handled in the following manner:

- The interrupt controller checks if the received interrupt type is enabled via the `mie` and `mstatus` CSRs.
- If it is, at the start of the next ID stage execution (ID1):
    - An interrupt signal is asserted and propagated to the relevant pipeline stages. This in particular triggers a redirection in execution.
    - The `mcause` CSR is loaded with the relevant triggered code.
    - The interrupt jump target is fed by the `mtvec` CSR.
    - The `mstatus.mie` bit is stored to the `mstatus.mpie` and cleared.
    - The exception program counter is captured to the `mepc` CSR: If the ID stage instruction at the moment of the interrupt is a Jump or a Branch instruction, its program counter is captured. Otherwise, the program counter of the instruction at the front of the Prefetch Buffer is captured.

      The `mret` instruction is used for resuming program execution. Execution is resumed in the following manner:
- The `mstatus.mie` bit is restored according to the value in `mstatus.mpie`.
- The resumption jump target is derived from the `mepc` CSR.

Module Implementation

Before detailing the specific combinational logic and state machines of each pipeline stage, the following figure provides a high-level microarchitectural roadmap of the processor. This block diagram illustrates the core's three-stage, in-order pipeline, its interface with the external APB memory router, and the integration of the embedded AES-128 accelerator.



fetch_unit

Purpose

```
Implement the IF stage.
Fetch the next instruction from IMEM and buffer it if the
downstream stage is not ready. Maintains and redirects the PC.
Predicts branches.
```

Behaviour

Nominal Flow

- If the prefetch buffer is not full, the module starts an APB read transfer with IMEM to fetch the next instruction.

- In the cycle when the read transfer finishes, if the prefetch buffer is empty, the instruction and its PC are outputted in `inst_o` and `pc_o`. If the downstream stage is not ready to accept the instruction (as indicated by `ready_i = 0`), the instruction and its pc are stored in a prefetch buffer.
- While the prefetch buffer is not empty, the next instruction and PC are outputted from it in `inst_o` and `pc_o`, and fetched instructions are stored in it.
- When an instruction is ready to be outputted (either directly from the read transfer or the prefetch buffer) `valid_o` is held high.
- Static branch predictor that predicts branch taken if the jump target is before the branch PC, and not taken otherwise (implemented in `branch_predictor_sbm`).
- When a jump instruction is decoded in the ID stage, or an interrupt is triggered, (as indicated by `jmp_i` and `interrupt_i`), the module updates PC to the address in `jmp_target_i`, starts a read transfer.
- When a branch instruction is decoded in the ID stage (as indicated by `branch_i`), the module updates PC based on the `branch_predictor_sbm` output, stores the alternative PC and switches to speculative fetching mode.
  The module reverts to normal fetching mode when the branch resolves (as indicated by `branch_cmp_result_valid_i` signal coming from the EXE stage).
  If the branch was miss-predicted, the module updates PC to the stored alternative, de-asserts `valid_o` and asserts `misspredict_o`. Otherwise it continues non-speculative fetching.
- On any PC redirection, the prefetch buffer is invalidated.
- To avoid starting a redundant read transfer the module checks if the instruction in the instruction buffer is a branch or a jump and when a jump or a branch instruction is outputted from the instruction prefetch buffer, it doesn't start fetching the next instruction but waits for the jump target which will be available in the next cycle.

External Signals

Signal	Dir	Width	Short description
`clk`, `rst_n`	I	1	Clock, active-low reset
`ready_i`	I	1	Downstream stage can accept instruction and PC
`jmp_i`	I	1	ID decodes an unconditional jump; target on `jmp_target_i` Or an interrupt is triggered, indicated by `interrupt_i` asserted as well
`branch_i`	I	1	ID decodes a branch; target on `jmp_target_i`
`interrupt_i`	I	1	Interrupt triggered. Used to distinguish interrupt jmps.
`branch_cmp_result_valid_i`	I	1	Branch condition resolved in EXE stage
`branch_cmp_result_i`	I	1	Branch condition resolve result

Signal	Dir	Width	Short description
`stall_for_jmp_target_i`	I	1	Stalls instruction fetching when asserted (Can only asserted in ST_INIT_FETCH and only has an effect there)
`jmp_target_i`	I	32	Unconditional jump, branch or interrupt target address
`inst31_i`	I	1	Last bit of the last outputted instruction from ID stage. Used by the branch predictor
`valid_o`	O	1	IF has a valid instruction for ID
`misspredict_o`	O	1	IF detected wrong prediction, request flush
`inst_o`	O	32	Currently issued instruction
`pc_o`	O	32	Currently issued PC
`pc_next_o`	O	32	The PC of the instruction that will be issued on the next `ready_i`
`imem_apb`	I/O	—	Bus bundle up to top level

Submodules

Instance	Brief role
apb_controller	APB master that manages APB read transfers with IMEM.
branch_predictor_sbm	A static branch predictor based on branch direction.
`prefetch_buffer_sbm`	A buffer that stores fetched instructions downstream is not ready to accept yet

apb_controller

Port	Dir	Width	Connected signal	Purpose
clk, rst_n	I	1	Clock, active-low reset	Clock
start_i	I	1	imem_apb_start	Start a transfer.
dir_i	I	1	1'b0	Lock controller read-only.
write_size_i	I	cs_size	SIZE_W	Irrelevant port in read-only.
wdata_i	I	32	32'b0	Not used (read-only)
addr_i	I	32	imem_apb_fetch_address	Read address
apb	I/O	—	imem_apb	Bus bundle up to top level
ready_o	O	1	imem_apb_ready	Controller can start another transfer in the next cycle.
valid_o	O	1	imem_apb_valid	Read completes, rdata_o has valid data.
err_o	O	1	imem_apb_err	APB error
rdata_o	O	32	imem_apb_rdata	Fetched data

branch_predictor_sbm

Port	Dir	Width	Connected signal	Purpose
imm_sign_i	I	1	inst31_i	Sign bit of current branch
take_branch_o	O	1	take_branch	Predictor decision

prefetch_buffer_sbm

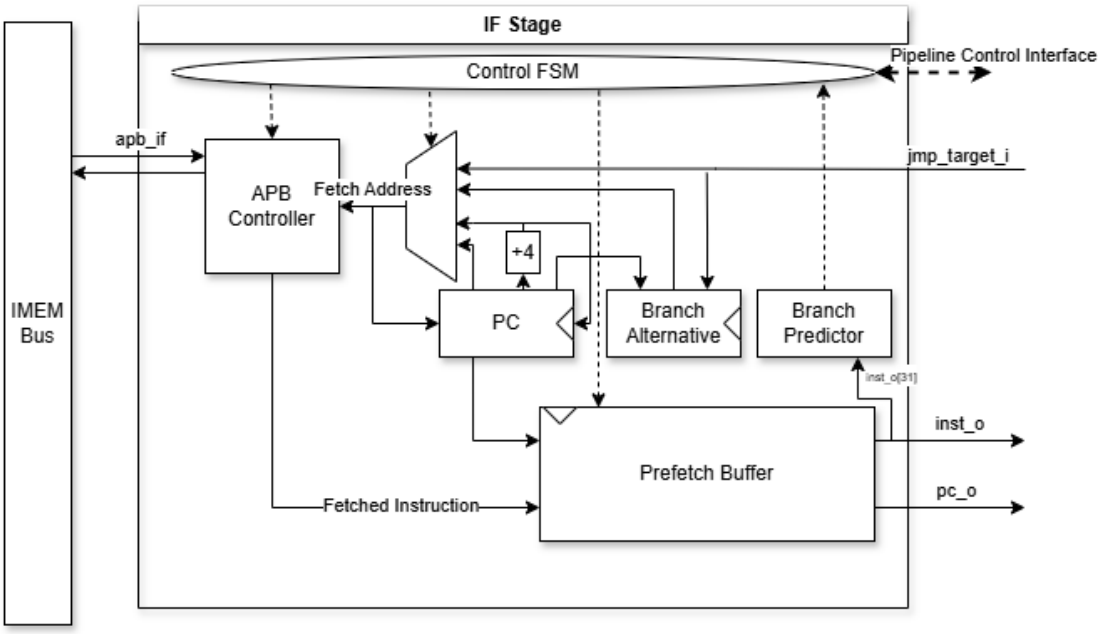
Port	Dir	Width	Connected signal	Behavior	IF-Stage usage
`inst_i`	I	32	`imem_apb_rdata`	Instruction word captured when `write_i` is 1.	Driven by APB read data on a fill; enqueued into buffer.
`pc_i`	I	32	`pc_current`	PC stored alongside `inst_i` at enqueue	The fetch PC for `inst_i`.
`enqueue_i`	I	1	`prefetch_enqueue`	Enqueue tail: store `inst_i` and `pc_i`.	Asserted on APB read fill, if the buffer is not empty or it is empty but the downstream stage is not ready.
`dequeue_i`	I	1	`prefetch_dequeue`	Dequeue head to outputs	Asserted when downstream stage is ready to accept the instruction and PC in `inst_o` and `pc_o`. (when the buffer is not empty)
`flush_i`	I	1	`prefetch_flush`	Invalidates the instructions stored in the buffer.	Asserted on redirect.
`inst_o`	O	32	`prefetch_inst`	Instruction at the head of the queue.	Outputted in IF's `inst_o` when the buffer is not empty
`pc_o`	O	32	`prefetch_pc`	PC of the Instruction at the head of the queue.	Outputted in IF's `pc_o` when the buffer is not empty

Port	Dir	Width	Connected signal	Behavior	IF-Stage usage
full_o	O	1	prefetech_full	Asserted when the buffer is full	not used #o
full_next_o	O	1	prefetech_full_next	Asserted when the queue will be full the next cycle	Moves IF stage to ST_FULL_BUFFER when asserted
empty_o	O	1	prefetch_empty	Asserted when the queue is emprty	When asserted, IF stage outputs instruction from the queue. Otherwise, it outputs instructions directly from the APB controller.

Implementation

High-level micro-architecture

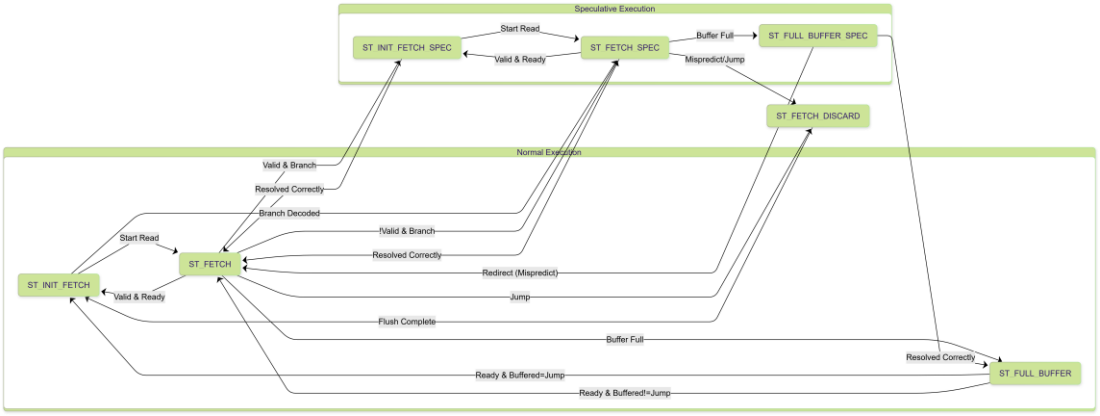
The following block diagram illustrates the data path and control flow between the submodules within the IF stage.



Key Internal Registers & Wires

Signal	Type	Width	Reset value	Description
`imem_apb_start`	wire	1	0	Pulse that requests an APB read from the IMEM controller. Asserted for one cycle in `ST_INIT_FETCH` (and `_SPEC`) states.
`imem_apb_fetch_address`	wire	32	-	Current fetch address placed on APB `PADDR`.
`redirect`	wire	1	-	Asserted if the next PC will not be `pc_current + 4` : on misspredict, on jump or branch. Triggers a `prefetch_buffer_sbm` flush.
`misspredict`	wire	1	-	Pulses when branch a prediction resolved to be wrong.
`interrupt`	wire	1	-	Pulses when an interrupt is recieved.
`pc_next`	wire	32	0	Contains `pc_current + 4`
`pc_current`	reg	32	`INIT_PC`	Tracks the address of the word currently being (or just) fetched.
`branch_alternative`	reg	32	0	`pc_current+4` or `jmp_target_i` cached in case branch mis-predicts.
`branch_taken`	reg	1	0	Stores speculative branch direction to compare with `branch_cmp_result_i`.
`jmp_target_requested`	reg	32	0	Stores the jump target for a jump that wasn't able to be acted on immediately (because it arrived mid fetch f.e.)

Control FSM



`ST_INIT_FETCH`

Purpose: Start a new instruction fetch.

Inputs inspected: `jmp_i`, `branch_i`, `take_branch`, `interrupt`, `stall_for_jmp_target_i`

Default actions

- `imem_apb_start = 1`.
- `inst_o, pc_o` driven from `preftech_buffer_sbm` if its not empty.
  Otherwise `inst_o = pc_o = X`
- `pc_current ← imem_apb_fetch_address`.
- `jmp_requested ← 0`

Exit table

Condition	Drives on transition	Register updates	Next state
`interrupt = 1`	`imem_apb_fetch_address = jmp_target_i`	`pc_current ← imem_apb_fetch_addess`	ST_FETCH
`jmp_i = 1`	`imem_apb_fetch_address = jmp_target_i`	`pc_current ← imem_apb_fetch_addess`	`ST_FETCH`
`branch_i & take_branch`	`imem_apb_fetch_address = jmp_target_i`	`branch_alternative ← pc_current+4` `branch_taken ← 1`	`ST_FETCH_SPEC`
`branch_i & !take_branch`	`imem_apb_fetch_address = pc_current+4`	`branch_alternative ←`	`ST_FETCH_SPEC`

Condition	Drives on transition	Register updates	Next state
		jmp_target_i branch_taken ← 0	
!stall_for_jmp_target	imem_apb_fetch_address = pc_current+4	pc_current ← imem_apb_fetch_addess	ST_FETCH

ST_FETCH

Purpose: Complete instruction fetch.

Inputs inspected: jmp_i, imem_apb_valid, branch_i

Default actions

- imem_apb_start = 0.
- imem_apb_fetch_address = pc_current
- inst_o, pc_o are driven from preftech_buffer_sbm if its not empty.
  Otherwise if imem_apb_valid then inst_o, pc_o are outputted from the APB controller.
  Otherwise inst_o = pc_o = X. valid_o is driven accordingly.
- If branch_i, The branch is not taken: branch_taken ← 0, branch_alternative ← jmp_target_i and transition is to the speculative version of the next state.

Exit table

Condition	Drives on transition	Register updates	Next state
jmp_i		jmp_requested ← 1 jmp_target_requested ← jmp_target_i	ST_FETCH_DISCARD
imem_apb_valid & !prefetch_empty & prefetch_full_next			ST_FULL_BUFFER or ST_FULL_BUFFER_SPEC
imem_apb_valid & !(!prefetch_empty &		pc_current ← pc_current + 4	ST_INIT_FETCH or ST_INIT_FETCH_SPEC

Condition	Drives on transition	Register updates	Next state
prefetch_full_next)			
!imem_apb_valid & branch_i			ST_FETCH_SPEC

ST_FULL_BUFFER

Purpose: Stop fetching when the instruction buffer is full until downstream is ready to receive it.

Inputs inspected: interrupt ready_i inst_in_buffer_branch_jmp

Default actions

- valid_o = 1
- inst_o = prefetch_inst
- `pc_o = prefetch_pc
- imem_apb_start = 0
- imem_apb_fetch_address = X

Exit table

Condition	Drives on transition	Register updates	Next state
interrupt	imem_apb_fetch_address = jmp_target_i	pc_current ← imem_apb_fetch_address	ST_FETCH
ready_i & inst_in_buffer_branch_jmp	–	pc_current ← pc_current + 4	ST_INIT_FETCH
ready_i & inst_in_buffer_branch_jmp	imem_apb_fetch_address = pc_next imem_apb_start = 1	pc_current ← imem_apb_fetch_address	ST_FETCH

_SPEC **states**

Perform as their non speculative counterparts and acts on branch resolve.

`ST_INIT_FETCH_SPEC`

Inputs inspected: `interruptmisspredictbranch_itake_branch`

Default actions

- `inst_o, pc_o` driven from `preftech_buffer_sbm` if its not empty and `valid_o` is driven accordingly.
  Otherwise `inst_o = pc_o = X`
- `imem_apb_start = 1`
- `pc_current ← imem_apb_fetch_address`

Exit table

Condition	Drives on transition	Register updates	Next state
`interrupt`	`imem_apb_fetch_address = jmp_target_i`	–	`ST_FETCH`
`misspredict`	`imem_apb_fetch_address = branch_alternative`	–	`ST_FETCH`
`branch_cmp_result_valid_i & !branch_i`	`imem_apb_fetch_address = pc_current`	–	`ST_FETCH`
`branch_cmp_result_valid_i & branch_i & !take_branch`	`imem_apb_fetch_address = pc_current`	`branch_alternative ← jmp_target_i`	`ST_FETCH_SPEC`
`branch_cmp_result_valid_i & branch_i & take_branch`	`imem_apb_fetch_address = jmp_target_i`	`branch_alternative ← pc_current`	`ST_FETCH_SPEC`
else	`imem_apb_fetch_address = pc_current`	–	`ST_FETCH_SPEC`

`ST_FETCH_SPEC`

Inputs inspected: `jmp_imisspredictimem_apb_validimem_apb_readyprefetch_full_nextbranch_ibranch_cmp_result_valid_i`

Default actions

- if `jmp_i` or `misspredict` then `inst_o = pc_o = X`.
  Otherwise `inst_o, pc_o` are driven from `preftech_buffer_sbm` if its not empty.
  Otherwise if `imem_apb_valid` then `inst_o, pc_o` are outputted from the APB controller.
  Otherwise `inst_o = pc_o = X`. `valid_o` is driven accordingly.
- `imem_apb_start = 0`
- `imem_apb_fetch_address = pc_current`

Exit table

- #o higher conditions have higher priority
  #continue down here after checking if the `prefetch_empty` condition is redundnet

Condition	Drives on transition	Register updates	Next state
`jmp_i`	–	`jmp_requested ← 1` `jmp_target_requested ← jmp_target_i`	`ST_FETCH_DISCARD`
`misspredict & !imem_apb_ready`	–	`pc_current ← branch_alternative`	`ST_FETCH_DISCARD`
`misspredict & imem_apb_ready`	–	`pc_current ← branch_alternative`	`ST_INIT_FETCH`
`imem_apb_valid & !prefetch_empty & !prefetch_full_next & !(branch_i || !branch_cmp_result_valid_i) & !branch_i`	–	`pc_current ← pc_next`	`ST_INIT_FETCH`
`imem_apb_valid & !prefetch_empty & !prefetch_full_next & branch_i`	–	`branch_taken ← 0` `branch_alternatice ← jmp_target_i`	`ST_INIT_FETCH_SPEC`
`imem_apb_valid & !prefetch_empty & !prefetch_full_next & (branch_i ||`	–	`pc_current ← pc_next`	`ST_INIT_FETCH_SPEC`

Condition	Drives on transition	Register updates	Next state
!branch_cmp_result_valid_i) & !branch_i			
imem_apb_valid & !prefetch_empty & prefetch_full_next & !(branch_i || !branch_cmp_result_valid_i) & !branch_i	–	–	ST_FULL_BUFFER
imem_apb_valid & !prefetch_empty & prefetch_full_next & (branch_i || !branch_cmp_result_valid_i) & !branch_i	–	–	ST_FULL_BUFFER_SPEC
imem_apb_valid & !prefetch_empty & prefetch_full_next & & branch_i	–	branch_taken ← 0 branch_alternatice ← jmp_target_i	ST_FULL_BUFFER_SPEC
imem_apb_valid & prefetch_empty & !prefetch_full_next & !(branch_i || !branch_cmp_result_valid_i) & !branch_i			

ST_FULL_BUFFER_SPEC

Inputs inspected: `ready_i`, `branch_cmp_result_valid_i`, `branch_cmp_result_i`, `branch_taken`, `branch_i`, `jmp_i`

Default actions

- `valid_o = 1`
- `inst_o = inst_buffer
- `pc_o = pc_current`

Exit table

Condition	Drives on transition	Register updates	Next state	
`branch_cmp_result_valid_i & (branch_taken != branch_cmp_result_i)`	`imem_apb_start = 1` `imem_apb_fetch_address = branch_alternative`	`pc_current ← imem_apb_fetch_address`	`ST_FETCH`	
`branch_cmp_result_valid_i & (branch_taken = branch_cmp_result_i) & inst_in_buffer_branch_jmp & (jmp_i | branch_i)` #o this shouldn't be possible?	`imem_apb_start = 0` `imem_apb_fetch_address = jmp_target`	`pc_current ← imem_apb_fetch_address`	`ST_INIT_FETCH`	
`!branch_cmp_result_valid_i & ready_i & inst_in_buffer_branch_jmp`	`imem_apb_start = 0` `imem_apb_fetch_address = pc_next`	`pc_current ← imem_apb_fetch_address`	`ST_INIT_FETCH`	
`!branch_cmp_result_valid_i & ready_i & !inst_in_buffer_branch_jmp & (jmp_i | branch_i)`	`imem_apb_start = 1` `imem_apb_fetch_address = jmp_target`	`pc_current ← imem_apb_fetch_address`	`ST_FETCH`	
`!branch_cmp_result_valid_i & ready_i & !inst_in_buffer_branch_jmp & !(jmp_i | branch_i)`	`imem_apb_start = 1` `imem_apb_fetch_address = pc_next`	`pc_current ← imem_apb_fetch_address`	`ST_FETCH`	

ST_FETCH_DISCARD

Purpose: Wait until a miss-predicted instruction fetch finishes and discard it.

Inputs inspected: `imem_apb_ready`

Default actions

- `valid_o = 0`

- `inst_o, pc_o = X
- `imem_apb_start = 0`
- `imem_apb_fetch_address = pc_current`

Exit table

Condition	Drives on transition	Register updates	Next state	
`imem_apb_ready`	–	–	`ST_INIT_FETCH`	

#continue

Description

If `branch_cmp_result_valid_i = 1`:

- If `branch_taken != branch_cmp_result_i` (miss predicted):
  - Drives `imem_apb_fetch_address` to `branch_alternative`
  - Transitions to `ST_FETCH` (continue current fetch which is non-speculative)
- Else (predicted correctly)
  - If `branch_i = 1` (another branch):
    - Drives `imem_apb_fetch_address` to `pc_next` or `jmp_target_i`, based on branch prediction.
    - Transitions to `ST_FETCH_SPEC` (continue current fetch which is speculative)
  - Else:
    - Transition to `ST_FETCH`

State	Description	Exit conditions
`ST_INIT_FETCH`	Performs the first cycle of the IMEM APB read transfer using `imem_apb_controller` by setting `imem_apb_fetch_address` and asserting `imem_apb_start`. Decides the fetch address (`imem_apb_fetch_address`) based on `jmp_i`, `branch_i`, and `take_branch`: If `jmp_i` or `branch_i && take_branch`: - Drives `imem_apb_fetch_address`	`branch_i = 0` -> `ST_FETCH` `branch_i = 1` -> `ST_FETCH_SPEC`

State	Description	Exit conditions
	to jmp_target_i. - If branch_i = 1 - - Stores pc_current + 4 in branch_alternative -- Transitions to ST_FETCH_SPEC (continue fetch speculatively) - Else transitions to ST_FETCH (continue fetch) Else: - Drives imem_apb_fetch_address to pc_current + 4. - Transitions to ST_FETCH (continue fetch) Stores imem_apb_fetch_address to pc_current.	
ST_FETCH	Waits for the IMEM APB read transfer to complete as indicated by imem_apb_valid. On transfer completion, If ready_i = 1: - Drives inst_o and pc_o with the fetched word. - Transitions to ST_INIT_FETCH (start fetch) Else - Stores the fetched word in inst_buffer - Transitions to ST_FULL_BUFFER. (stop fetching, wait for buffer to clear)	imem_apb_valid = 1 && ready_i = 0 -> ST_FULL_BUFFER imem_apb_valid = 1 && ready_i = 1 -> ST_INIT_FETCH
ST_FULL_BUFFER	Drives inst_o and pc_o with the fetched word. Waits for the downstream module to be ready to receive the buffered instruction. Sets the fetch address to pc_current + 4. If ready_i = 1: - If inst_in_buffer_branch_jmp = 0 it performs the first cycle of the IMEM APB read	ready_i = 1 && inst_in_buffer_branch_jmp = 0 -> ST_FETCH ready_i = 1 && inst_in_buffer_branch_jmp = 0 -> ST_INIT_FETCH

State	Description	Exit conditions
	transfer and transitions to `ST_FETCH` (complete fetch) - Else transitions to `ST_INIT_FETCH` (start fetching)	
`_SPEC` states	Performs as their non speculative counterparts and acts on branch resolve.	
`ST_INIT_FETCH_SPEC`	If `branch_cmp_result_valid_i = 1`: - If `branch_taken != branch_cmp_result_i` (miss predicted): - - Set `imem_apb_fetch_address` to `branch_alternative` - - Transition to `ST_FETCH` - Else If `branch_i = 1` (another branch): - -	
`ST_FETCH_SPEC`		
`ST_FULL_BUFFER_SPEC`		

decode_unit

Purpose

```
Implement the ID stage.
Decode, generate control signals, immediate, jump targets.
Execution first and second instruction execution steps. Detect
hazards, stall and forward.
```

Behaviour

Nominal Flow

- When downstream is ready and the upstream module output is valid, store instruction and PC from inputs into registers.
- Generate control signals according to the stored instruction, for both this stage and downstream stage. Calculate and output the jump target if relevant.
- Detect hazards. If a hazard is detected, avoid it by enabling a forwarding mechanism or stalling the pipe.

- Enable output registers and output data and control signals according to the generated control signals. Issue the data downstream over two cycles.

External Signals

Signal	Dir	Width	Short description
`clk`, `rst_n`	I	1	Clock, active-low reset
`ready_i`	I	1	EXE stage can accept instruction and PC
`valid_i`	I	1	IF stage output is valid
`exe_dmem_apb_ready_d_i`	I	1	Asserted when the DMEM APB interface in EXE stage is ready
`misspredict_i`	I	1	Asserted by the IF stage on misprediction. Triggers a flush.
`interrupt_i`	I	1	Asserted by the EXE stage on interrupt. Triggers execution redirection.
`accel_ready_i`	I	1	Asserted when the accelerator in the EXE stage is ready.
`inst_i`	I	32	Input instruction from IF stage.
`pc_i`	I	32	PC of inputted instruction from IF stage.
`reg32_i`	I	32	Register file read data.
`interrupt_jmp_target_i`	I	32	Valid when `interrupt_i` is asserted and contains the interrupt jump target.
`mepc_i`	I	32	The bits stored in the MEPC CSR. Used as a jump target in the implementation of MRET.

Signal	Dir	Width	Short description
exe_regfile_write_half_i	I	16	Forwarded value of the half currently written to the register file in the EXE stage.
cs_exe_o	O		Control signals for EXE stage
jmp_o	O	1	Triggers a jump in the IF stage when asserted.
branch_o	O	1	Triggers a branch in the IF stage when asserted.
ready_o	O	1	Instruction decode and issuing finished, ID stage is ready receive the next instruction.
valid_o	O	1	Control signals and output data for EXE stage are valid
dmem_load_bypass_o	O	1	Bypass to EXE stage - start DMEM APB read transfer.
exe_first_cycle_o	O	1	First cycle signal for EXE stage.
fetch_stall_for_jmp_target_o	O	1	Asserted when the IF stage should pause fetching until the correct jump target is outputted. (jump target may not be ready in case of pipe stalls)
resume_execution_from_dec_inst_o	O	1	After an handling an interrupt, execution should resume from the instruction currently in ID stage.
ready_for_interrupt_o	O	1	Asserted when the ID stage is ready to receive an interrupt, which is when it is not in the middle of issuing an instruction.

Signal	Dir	Width	Short description
accel_rf_write_o	O	1	When asserted, the register file stores the data in the accel output.
lsu_store_addr_o	O	32	Store address for LSU
lsu_load_addr_bypass	O	32	Bypass to EXE stage, Load address for LSU
jmp_target_o	O	32	Jump target for IF stage
alu_a_o	O	16	ALU input a data register for EXE stage
alu_b_o	O	16	ALU input b data register for EXE stage
wb_o	O	16	WB data for EXE stage
rd_o	O	5	Register file write point index
rs32_o	O	5	Register file 32 bit read port index
inst31_o	O	1	The last bit of the currently decoded instruction
csr_addr_o	O	12	CSR address register for EXE stage
pc_o	O	32	Program counter of currently decoded instruction

Submodules

Submodule	Brief role
control_unit	Generates the control signals for the current instruction
imm_gen_sbm	Generates the immediate for the current instruction

Submodule	Brief role
`serializer_32_to_16`	Serializes 32 bit word to two 16 bits halves in two cycles.

control_unit
Purpose:

- Generate the control signals for the current instruction.

Module Interface

Port	Dir	Width	Connected signal	Purpose
`clk`, `rst_n`	I	1	`clk`, `rst_n`	Clock, active-low reset
`first_cycle`	I	1	`first_cycle`	First decode cycle indicator
`opcode_i`	I	7	`opcode`	Currently decoded instruction's opcode field.
`funct3_i`	I	3	`funct3`	Currently decoded instruction's funct3 field.
`funct7_i`	I	7	`funct7`	Currently decoded instruction's funct7 field.
`funct12_least_5_i`	I	5	`rs2`	5 least significant bit of funct12 field (the 7 most significant ones are funct7) Used in some SYSTEM instructions.
`exe_sel_d_i`	I	#w	`cs_exe_o.sel`	EXE stage `sel` control signals. Those are sampled in the control unit and used as default assignment for the next controls output.
`cs_o`	O	#w	`cs`	All control signals for this cycle.

imm_gen_sbm
Purpose:
Generates the immediate for the current instruction.

Module Interface

Port	Dir	Width	Connected signal	Purpose
`inst_type_i`	I	#w	`cs.dec.sel.inst_type`	Currently decoded instruction's type. Used to determine the immediate format.
`inst_i`	I	32	`inst`	Currently decoded instruction.
`imm_o`	O	32	`imm`	Generated immediate

serializer_32_to_16

Purpose

Serializes a 32-bit word into two 16-bits halves.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`data_i`	I	32	32-bit word to serialize.
`start_i`	I	1	Start serialize signal. When asserted, the *first* (*) half of the input will be propagated to the output the same cycle, and the second half will be propagated in the next cycle. If `SAMPLE_ON_START = 1`: The second half will be of the output of the previous cycle. If `SAMPLE_ON_START = 0`: The second half will be of the output of the same cycle.
`start_half_i`	I	1	(*) Determines which half (lower or upper) is the *first* outputted half.
`data_o`	O	16	serialized 16-bit half words.

Parameters

- **SAMPLE_ON_START**:
  - If `1`: On `start_i` assertion, `data_i`'s second half gets sampled and that sampled half will be outputted on the next cycle.
    For example, this mode is used in `decode_unit` to:

        - Forward one half of the register file's output, and sample the other half in the first execution step.
        - Forward the other half in the second execution step, while the register file's output is used for another task.
    - If `0`: `start_i` determines if the first or second half of `data_i` propagates this cycle.

Instantiations

- **Instance 1**: `sample_ser_alu_a`
    - **Role**: Feeds the ALU Operand A.
    - **Parameter**: `SAMPLE_ON_START = 1
    - **Key Connections**
        - `data_i` Register File read port.
        - `data_oalu_a_o`*Combinatorically*.
        - `start_i` Asserted during the second ID stage execution cycle (which is the first EXE stage execution cycle)
- **Instance 2: `no_sample_ser_alu_b_wb`
    - **Role**: Feeds the ALU Operand B output register (`alu_b_o`) or the WB output register (`wb_o`) .
    - **Parameter**: `SAMPLE_ON_START = 0
    - **Key Connections**
        - `data_i` Depends on the decoded instruction. Can be either: immediate value, Register File output from the first ID stage execution stage, PC, PC plus immediate value or Register File output plus immediate value.
        - `data_oalu_a_o` or `wb_o`*Sequentially*.
        - `start_i` Asserted during the first ID stage execution cycle.

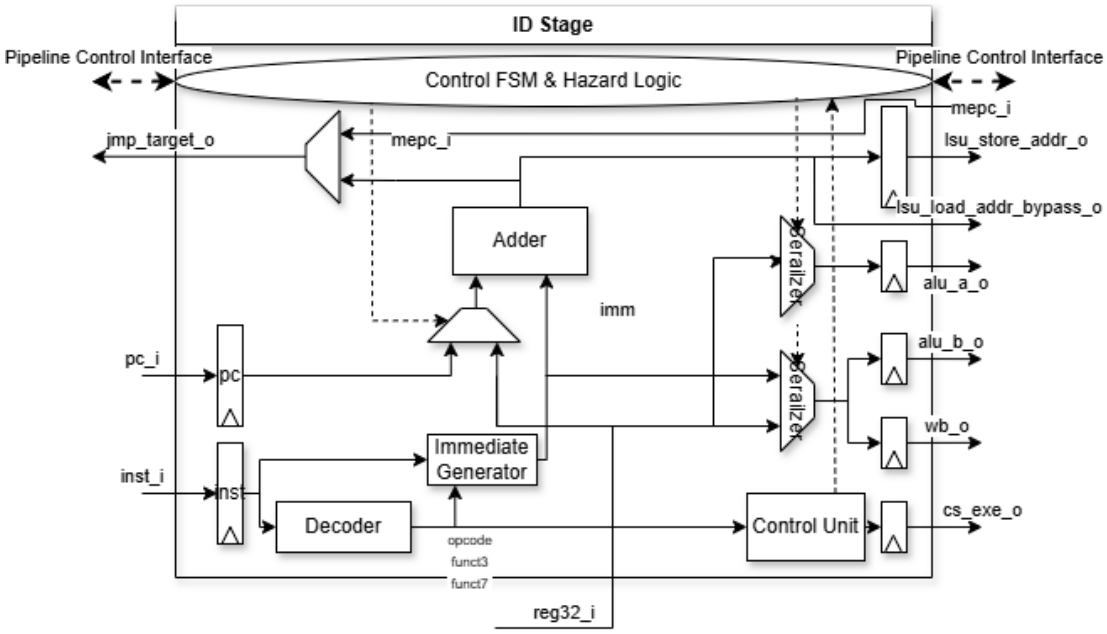
Implementation

High-level Micro-Architecture

The following block diagram illustrates the data path and control flow within the ID stage. In addition to the instantiated submodules, the module logic consists of:

- **Decode logic** that parses the current instruction (`opcode`, `funct3`, `funct7`).
- **Hazard detection and forwarding logic** to stall the pipe or forward data when dependencies are detected.
- **A dedicated Adder** used for calculating jump targets, ALU operand B, and LSU load addresses.
- **Output Registers** that latch the data and control signals to feed the EXE stage.

- **A Control FSM** that sequences the 16-bit split-cycle execution.



Key Internal Registers & Wires

Wires

Signal	Width	Description
`reg32_used_in_first_cycle`	1	Asserted if the Register File read port is used in the first cycle, based on the control signals.
`full_read_after_write_rs1_hazard`	1	Asserted on FRAW1 hazard.
`full_read_after_write_rs2_hazard`	1	Asserted on FRAW2 hazard.
`half_read_after_write_hazard`	1	Asserted on HRAW2 hazard.
`store_load_hazard`	1	Asserted on SL hazard.
`trigger_stall_when_ready`	1	Triggers a stall on the first ID stage execution cycle. Asserted on hazard detection.
`stall_one_cycle`	1	Stalls the pipe when asserted by de-asserting `issue`. Asserted on the first ID stage execution cycle if

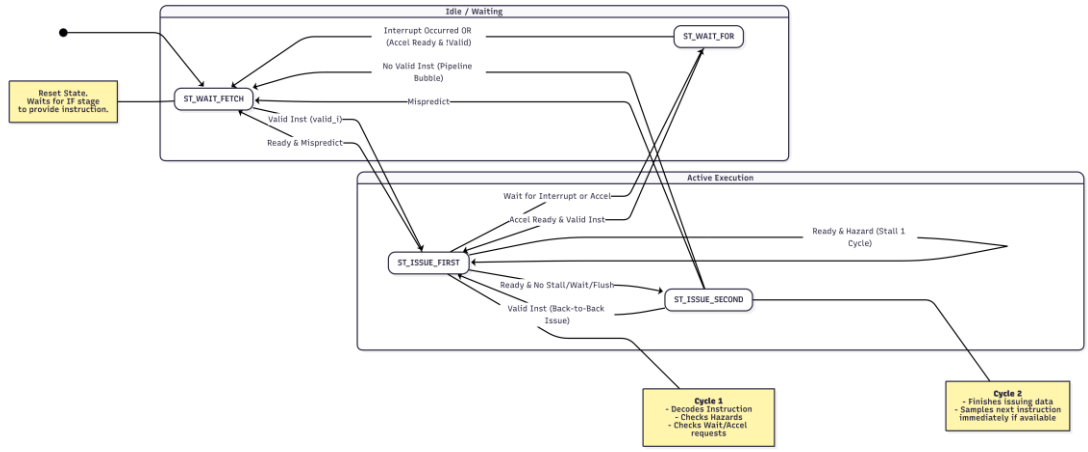
Signal	Width	Description
		`trigger_stall_when_ready` is asserted.
`signal_jmp_issue_first`	1	Asserted to signal a jump in the FSM state `ST_ISSUE_FIRST`. Nominally asserted on interrupt or by a a jump control signal.
`signal_jmp_wait_fetch_or_interrupt`	1	Asserted to signal a jump in the FSM states `ST_WAIT_FETCH`, `ST_WAIT_FOR`. Nominally asserted on interrupt. Is sampled to reg `signaled_jmp` that prevents additional jump signaling when `1`.
`signal_branch_issue_first`	1	Asserted to signal a branch in the FSM state `ST_ISSUE_FIRST`. Nominally asserted on a branch control signal. Is sampled to reg `signaled_branch` that prevents additional branch signaling when `1`.
`inst_jmp_or_branch`	1	Asserted if the current decoded instruction is a jump or a branch instruction.
`jmp_target`	32	Carries the jump target address. On jump and branch instructions, is fed by the ID Adder. On MRET instruction is fed by CSR MEPC.
`reg32_i_first_cycle`	32	Carries FRAW2 Hazard-corrected read data. Driven by the Register File port in the first cycle and by the holding register `reg32_i_first_cycle_d` in subsequent cycles.
`first_cycle`	1	Asserted during the first ID execution step.

Signal	Width	Description
`issue`	1	Asserted when valid data and control signals are issued to the EXE stage.

Registers

Signal	Width	Reset value	Description
`inst`	32	NOP	Holds the currently decoded instruction.
`pc`	32	0	Hold the PC of the currently decoded instruction.
`signaled_jmp`	1	0	Holds `1` if a jump was already signaled for the current instruction or interrupt.
`signaled_branch`	1	0	Holds `1` if a branch was already signaled for the current instruction or interrupt.
`reg32_i_d`	32	0	Holds the FRAW2 Hazard-corrected read data for the second ID execution step.

Control FSM



`ST_ISSUE_FIRST`

Purpose

Issue the first batch of data and control signals to the EXE stage.

Inputs inspected:

`misspredict_iready_i` stall_one_cycle `issue` cs.dec.en.wait_for_interrupt cs.dec.en.wait_for_accel`

Default actions

- `valid_o` = `!misspredict_i`
- `ready_o` = `0`
- `inst31_o` = `inst[31]`
- `first_cycle` = `1`
- `lsu_load_addr_bypass_o` = `0`
- `dmem_load_bypass_o` = `0`
- `rs32_o` = If the Register File is used in the executed instruction, it selects `rs1` for Jumps/Load Bypass/Stores otherwise `rs2`.
- `fetch_stall_for_jmp_target_o` = 1 if current instruction a jump and there is a FRAW1 hazard stall.
- `accel_rf_write` = `0`
- `jmp_target_o` = `jmp_target` if current instruction is a branch or if it is a jump and these is no fetch stall or no interrupt. In case of an interrupt `jmp_target_o` = `interrupt_jmp_target_i`

Internal State Updates

- `signaled_jmpsignal_jmp_issue_first`
- `signaled_branchsignal_branch_issue_first`

Exit table

Condition	Drives on transition	Register updates	Next state
`cs.dec.en.wait_for_interrupt | cs.dec.en.wait_for_accel`	`issue = 0` `valid_o = 0	-	`ST_WAIT_FOR`
`ready_i & misspredict_i`	`issue = 0` `valid_o = 0`	-	`ST_WAIT_FETCH`
`ready_i & !misspredict_i & !stall_one_cycle`	`issue = 1` `valid_o = 1` `dmem_load_bypass_o = cs.dec.en.dmem_load_bypass` `lsu_load_addr_bypass_o =`	-	`ST_ISSUE_SECOND`

Condition	Drives on transition	Register updates	Next state
	`cs.dec.en.dmem_load_bypass ? add_out : 0`		

`ST_ISSUE_FIRST`

Purpose

Issue the second batch of data and control signals to the EXE stage. Sample next instruction if it is ready.

Inputs inspected:

`misspredict_ivalid_ivalid_o_d`

Default actions

- `issue` = `!misspredict_i`
- `valid_o = 1`
- `ready_o = 1`
- `rs32_o` is assigned according to the instruction currently executed.

Internal State Updates

- `signaled_jmp0`
- `signaled_branch0`

Exit table

Condition	Drives on transition	Register updates	Next state
`misspredict_i`	-	-	`ST_WAIT_FETCH`
`valid_i`	-	`instinst_i pcpc_i`	`ST_ISSUE_FIRST`
else	-	-	`ST_WAIT_FETCH`

`ST_WAIT_FETCH`

Purpose

Wait for the next instruction to be ready.

Inputs inspected:

`interrupt_imisspredict_ivalid_isignal_jmp_wait_fetch_or_interrupt`

Default actions

- `issue = 0`
- `valid_o` is held until EXE stage finishes executing the issued instruction.
- `jmp_target_o = interrupt_jmp_target_i` on interrupt.

Internal State Updates

- `signaled_jmpsignal_jmp_wait_fetch_or_interrupt` if the next instruction is not ready and `0` otherwise.

Exit table

Condition	Drives on transition	Register updates	Next state
`valid_i`	-	`instinst_i` `pcpc_i`	`ST_ISSUE_FIRST`

`ST_WAIT_FETCH`

Purpose

Wait for an event.

Inputs inspected:

`interrupt_iaccel_ready_isignal_jmp_wait_fetch_or_interrupt`

Default actions

- `issue = 0`
- `ready_o = 0`
- `valid_o = 0`
- `jmp_target_o = interrupt_jmp_target_i` on interrupt.

Internal State Updates

- `signaled_jmpsignal_jmp_wait_fetch_or_interrupt` if the next instruction is not ready and `0` otherwise.

Exit table

Condition	Drives on transition	Register updates	Next state
`interrupt_i`	-	-	`ST_WAIT_FETCH`
`cs.dec.en.wait for_interrupt & accel_ready_i & valid_i`	-	`instinst_i pcpc_i`	`ST_ISSUE_FIRST`
`cs.dec.en.wait for_interrupt & accel_ready_i & !valid_i`	-	-	`ST_WAIT_FETCH`

exe_mem_wb_stage

Purpose

```
Implement the EXE stage.
Do arithmetics. DMEM read and write. Register File write back.
Encryption acceleration. CSR read and writes. Interrupt control.
```

Behaviour

The EXE stage servers as the processor's primary computational engine, executing operations dictated by the control signals received from the ID stage. Unlick a traditional pipeline stage that fetches its own data, this stage operates on pre-fetched operands (`alu_a_i`, `alu_b_i`, `wb_i`) and control information provided directly by the ID stage.

Upon the assertion of `first_cycle`, the stage captures these inputs and routes them to the appropriate functional units:

- **ALU Operations:** Standard arithmetic operands are fed directly to the ALU. For CSR instructions, the stage internally multiplexes the ALU inputs to select between the ID-supplied register values and local immediate extenders or CSR read data.
- **Memory Access:** The LSU is triggered if the control signal indicate a memory operation. It managers the APB interface protocol to perform reads or writes to the Data Memory.
- **Accelerators & CSRs**: Dedicated enable signals selectively activate either the AES core or the CSR bank logic only when required by the current instruction, preventing unnecessary switching activity.

Execution is managed through a structurally enforced multi-cycle flow to accommodate the architecture's 16-split data path. The `ready_o` signal is logically gated, ensuring the stage remains active for at least two cycles to process the lower and upper halves of the operands. The pipeline is stalled beyond this minimum duration only if longer operations such as an incomplete APB memory transfer prevent the stage from completing.

External Signals

Signal	Dir	Width	Short description
clk, rst_n	I	1	Clock, active-low reset
first_cycle	I	1	Indicates if the current execution step is the first one.
valid_i	I	1	ID stage output is valid
dmem_load_bypass_i	I	1	Triggers a DMEM read request as part of Load instruction execution steps.
cs_i	I		Control signals outputted by the control unit according to the executed instruction.
interrupt_req_ext_i	I	1	An interrupt is requested by an external module.
dec_ready_for_interrupt_i	I	1	Indicates that ID stage can be interrupted this cycle.
accel_rf_write_i	I	1	When asserted the Accelerator output is written to the Register File.
dmem_apb	IO		DMEM APB interface.
lsu_store_addr_i	I	32	DMEM address for store instructions.
load_addr_bypass_i	I	32	DMEM address for load instructions.
alu_a_i	I	16	ALU operand A data.
alu_b_i	I	16	ALU operand B data.
wb_i	I	16	Register File write back data.
rd_i	I	5	Register File write back destination register index.
rs32_i	I	5	Register File read source register index.

Signal	Dir	Width	Short description
csr_addr_i	I	12	CSR read/write address.
fetch_pc_next_i	I	32	PC of currently fetched instruction. Used as a candidate for the return address after an interrupt.
dec_pc_i	I	32	PC of currently decoded instruction. Used as a candidate for the return address after an interrupt.
dec_resume_execution_from_dec_inst_i	I	1	Determines whether execution after an interrupt will be resumed from the address that was on dec_pc_i or fetch_pc_next_i in the moment of the interrupt.
ready_o	O	1	EXE stage is ready to input a new instruction controls and data the next cycles.
dmem_apb_ready_d_o	O	1	Indicates whether the DMEM APB interface was ready at the end of the last cycle. Used for the detection of SL hazards.
interrupt_o	O	1	Triggers an interrupt in the core.
reg32_o	O	32	Register File read port data.
cmp_result_o	O	1	Compare result.
cmp_result_valid_o	O	1	Compare result is valid.
shift_bigger_then_16_o	O	1	Asserted on a shift instruction, when the shift amount is bigger then 16. This signals ID stage to not forward the lower half of the operand.
interrupt_jmp_target_o	O	32	The address IF stage should fetch from on an interrupt.
mepc_o	O	32	CSR MEPC data.

Signal	Dir	Width	Short description
`regfile_write_half_o`	O	16	The value of the half currently written to the Register File. Used for forwarding.
`accel_ready_o`	O	1	Accelerator is ready to input new data. Used in ID stage for the implementation of the Wait For Accel instruction.

Submodules

`load_store_unit`

Purpose:
Interface the Core and DMEM.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`first_cycle`	I	1	First EXE cycle indicator
`start_i`	I	1	Start a read or write transfer.
`dir_i`	I	1	Indicates if the started transfer is read or write.
`size_i`	I	1	Determines the read or write size : word, half word or byte.
`load_ext_i`	I	1	Determines the extension type: zero or sign extension.
`dmem_apb`	IO		DMEM APB interface.
`reg1_i`	I	32	Input write data.
`addr_i`	I	32	Read or write DMEM address.
`half_o`	O	1	Indicates what half of loaded data is currently being outputted.
`apb_ready_o`	O	1	Indicates if the APB interface is ready for another transfer.

Port	Dir	Width	Purpose
`ready_o`	O	1	Indicates if the LSU is done and ready for another transfer.
`valid_o`	O	1	Indicates if the LSU outputs are valid.
`err_o`	O	1	Signals an error in the APB interface.
`ldata_o`	O	16	Half of read data output.

alu_sbm

Purpose
Do most of the Core's arithmetics.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`first_cycle`	I	1	First EXE cycle indicator
`op_i`	I		Operation selector
`cmp_req_i`	I	1	A comparison between operands is requested. Will lead to `cmp_result_valid_o` being asserted and a valid output in `cmp_result_o`.
`cmp_flip_i`	I	1	Flips the comparison result when asserted.
`a_i`	I	16	Operand A data.
`b_i`	I	16	Operand B data.
`shift_bigger_then_16_o`	O	1	On shift operation, it signals if the shift amount is bigger than 16.
`result_o`	O	16	Operation result.
`cmp_result_o`	O	1	Comparison result.

Port	Dir	Width	Purpose
`cmp_result_valid_o`	O	1	Comparison result is valid.

regfile_sbm

Purpose

Implements the RV32I architectural register file. It serves as the central data storage for the processor, providing read and write access for both standard integer instructions and the embedded accelerator.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`write_i`	I	1	Enable write of data from `write_data_i` to destination register index `rd_i`.
`rs32_i`	I	5	Read source register index.
`rd_i`	I	5	Write destination register index.
`write_data_i`	I	16	Write data input.
`rd_h_sel_i`	I	1	Selects which half of the destination register is written.
`accel_di`	I	128	Accelerator data input. Is connected to registers `x28-x31`
`accel_write_en_i`	I	1	Enable write of data from `accel_di` to destination registers `x28-x31`.
`rs32_o`	O	32	Read data from source register index `rs32_i`.
`accel_do`	O	128	Outputs data from the accelerator's registers`x28-x31`.

aes128_core

Purpose

Accelerate AES encryption and decryption.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`load_key_i`	I	1	Set data in `data_i` to be the AES encryption key.
`start_enc_i`	I	1	Start encryption of data in `data_i`
`start_dec_i`	I	1	Start decryption of data in `data_i`
`data_i`	I	128	Data input for key loading, encryption and decryption.
`data_o`	O	128	Encrypted/Decrypted data output.
`ready_o`	O	1	Accelerator is ready to start another encryption/decryption.

csr_sbm

Purpose

Store and handle CSR logic.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`write_i`	I	1	Write data in `write_data_i` to half (determined by `h_sel_i`) of the CSR in address `csr_addr`.
`h_sel_i`	I	1	Determine write target half.
`interrupt_req_ext_i`	I	1	External interrupt request. The value is directly wired to bit 11 of CSR MIP.
`interrupt_req_aes_i`	I	1	AES Accelerator interrupt request. The value is directly wired to bit 16 of CSR MIP.

Port	Dir	Width	Purpose
addr_i	I	12	General Read/Write CSR address.
write_data_i	I	16	General CSR input write data.
mepc_write_i	I	1	Write data in mepc_write_data_i to CSR MEPC.
mepc_write_data_i	I	32	MEPC CSR input write data
mcause_write_i	I	1	Write data in mcause_write_data_i to CSR MCAUSE.
mcause_write_data_i	I	32	MCAUSE CSR input write data
store_clear_mstatus_mie_i	I	1	On assertion, CSR MSTATUS.MIE (bit 3) is written to MSTATUS.MPIE (bit7) and CSR MSTATUS.MIE is reset. Is asserted on an interrupt to prevent unwanted nested interrupts.
restore_mstatus_mie_i	I	1	On assertion MSTATUS.MPIE (bit7) is written to CSR MSTATUS.MIE (bit 3). Used for the implementation of MRET instruction.
valid_o	O	1	CSR output is valid. (#o Currently unused)
read_data_o	O	16	General CSR Read data output.
mstatus_mie_o	O	1	CSR MSTATUS.MIE bit. Used in the interrupt controller.
mepc_o	O	32	CSR MEPC data output.
mie_o	O	32	CSR MIE data output.
mip_o	O	32	CSR MIP data output.
mtvec_o	O	32	CSR MTVEC data output.

interrupt_sbm

Purpose

Manages interrupts triggering logic and updates the relevant CSRs to handle the trap state.

Module Interface

Port	Dir	Width	Purpose
`clk`, `rst_n`	I	1	Clock, active-low reset
`fetch_pc_next_i`	I	32	IMEM address of the next instruction that is supposed to be outputted from the IF stage. A candidate to be the interrupt return address.
`dec_pc_i`	I	32	IMEM address of the instruction in ID stage. A candidate to be the interrupt return address.
`dec_resume_execution_from_dec_inst_i`	I	1	Determines if `fetch_pc_next_i` or `dec_pc_i` will be the interrupt return address in case of an interrupt.
`dec_ready_for_interrupt_i`	I	1	ID stage is ready to receive an interrupt.
`mip_i`	I	32	CSR MIP data.
`mie_i`	I	32	CSR MIE data.
`mtvec_i`	I	32	CSR MTVEC data.
`mstatus_mie_i`	I	1	CSR MSTATUS.MIE bit.
`mepc_write_o`	O	1	CSR MEPC write enable.

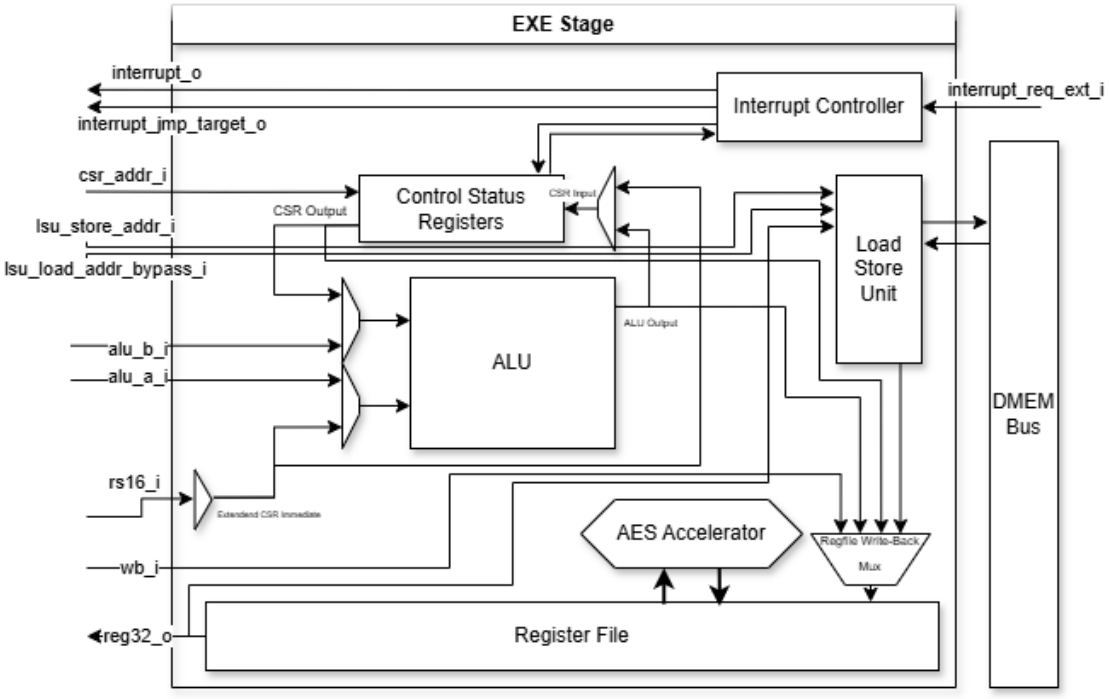
Port	Dir	Width	Purpose
`mepc_write_data_o`	O	32	CSR MEPC write data.
`mcause_write_o`	O	1	CSR MCAUSE write enable.
`mcause_write_data_o`	O	32	CSR MCAUSE write data.
`store_clear_mstatus_mie_o`	O	1	Trigger MSTATUS.MIE store to MSTATUS.MPIE and clear.
`interrupt_jmp_target_o`	O	32	Interrupt jump target.
`interrupt_o`	O	1	Triggers an interrupt when asserted.

Implementation

High-level micro-architecture

The following block diagram illustrates the data path routing between the submodules within the EXE stage. Because the execution sequencing is controlled entirely by the ID stage (via the `first_cycle` signal and control struct), this stage does not contain a dedicated FSM. Instead, it consists of the instantiated submodules and extensive multiplexing "glue" logic:

- **ALU Input Multiplexers:** Route the A and B operands into the ALU, selecting between the ID stage inputs (`alu_a_i`, `alu_b_i`) and CSR/Immediate data.
- **Register File Write-Back Mux:** Selects the final 16-bit result to be written back to the Register File. The source can be the ALU output, the LSU read data, the CSR read data, or a direct pass-through from the ID stage (`wb_i`).
- **CSR Write Mux:** Routes either the ALU output, ALU operand A, or an immediate value to the CSR write data port.
- **Cycle Tracking Logic:** Simple sequential logic to track the two-cycle execution rhythm dictated by the ID stage.



Key Internal Registers & Wires

Wires

Signal	Width	Description
`first_two_cycles`	1	Logical OR of `first_cycle` and `first_cycle_d`. Gates valid output signals and Register File write enables to ensure data is only written during active execution cycles, not during stalls.
`regfile_write_data`	16	The final combinatorial output of the Write-Back Multiplexer. Routes data from the ALU, LSU, CSR, or direct pass-through (`wb_i`) to the Register File.
`transfer_start`	1	Triggers the LSU's APB interface. Combines standard memory execution triggers with the `dmem_load_bypass_i` signal to initiate memory transfers early and hide latency.
`alu_a_mux`	16	Multiplexes the ALU's A operand, selecting between the ID stage input (`alu_a_i`) and CSR read data.

Signal	Width	Description
`alu_b_mux`	16	Multiplexes the ALU's B operand, selecting between the ID stage input (`alu_b_i`) and extended immediate data (`csr_imm_ext`).
`csr_write_data`	16	The multiplexed data routed to the CSR block's write port. Selects between the ALU output, the raw `alu_a_i` operand, or a zero-extended immediate.
`interrupt`	1	Triggers an asynchronous interrupt in the core. Driven directly by the Interrupt Controller module.
`store_clear_mstatus_mie`	1	Triggers the `MSTATUS.MIE` bit to be backed up to `MSTATUS.MPIE` and then cleared. Asserted by the Interrupt Controller during trap entry.

Registers

Signal	Width	Reset value	Description
`first_cycle_d`	1	0	Delays the `first_cycle` signal by one clock cycle. Effectively flags the second half of the 16-bit execution phase, keeping enables (like Register File writes) active across both cycles.
`apb_ready_d`	1	0	Samples the `apb_ready_o` signal from the LSU at the end of the cycle. Forwarded to the ID stage (`dmem_apb_ready_d_o`) to resolve Store-Load hazards.

Instruction Execution Steps Description

The tables omit:

- IF stage execution cycles (Cycles 1 and 2) which are identical for all instructions.
- Instruction decoding, control signals generation and forwarding are done in the ID1,2 steps for all instructions.
- If relevant to the instruction, the Immediate value is generated in ID1,2.

Arithmetic Instructions

Register-Immediate Arithmetic Instructions

`addi rd, rs1, imm`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Buffer the first half of the Immediate for the EXE stage.	Buffer the second half of the Immediate for the EXE stage. Read register `rs1` data from the Register File, forward its first half and buffer its second half.	
EXE		Perform the arithmetic operation on first halves. Write back the result to the first half of register `rd` in the Register File.	Perform the arithmetic operation on second halves. Write back the result to the second half of register `rd` in the Register File.

Register-Immediate Arithmetic Instructions

`add rd, rs1, rs2`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Read register `rs2` data from the Register File, buffer its first half and store its second half.	Buffer the stored second register half for the EXE stage. Read `rs1` data from the Register File, forward its first half and buffer its second half.	
EXE		Perform the arithmetic operation on first halves. Write back the result to the first half of register `rd` in the Register File.	Perform the arithmetic operation on second halves. Write back the result to the second half of register `rd` in the Register File.

- Half ordering per instruction:
    - Lower half first: ADD/I, SUB, XOR/I, OR/I, AND/I.

- Upper half first: SLL/I, SRL/I, SRA/I, SLT/I, SLTU/I.

Load Instructions

`ld rd, imm(rs)`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Calculate load address (`rs + imm`) If the LSU is available, forward that load address and start a DMEM read transfer, else stall.		
EXE		If DMEM read transfer is done, write back its lower half to the lower half of register `rd` in the Register File and store its upper half. Else stall.	Write back the stored upper half to the upper half of register `rd` in the Register File.

Store Instructions

`sw rs1, imm(rs2)`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Calculate store address (`rs2 + imm`) and buffer it to EXE stage.		
EXE		Read register `rs1` from Register File, store it in a register and start a DMEM write transfer of it to the received store address.	While the transfer is not done, stall.

Jump Instructions

`jalr rd, rs, imm`
`jal rd, imm`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Calculate jump target address (either `pc + imm` or `rs + imm`) . Trigger a jump in the IF stage. Buffer lower half of `pc` for the EXE stage.	Buffer upper half of `pc` for the EXE stage.	
EXE		Calculate lower half of `pc + 4` and write it back to register `rd` in the Register File.	Calculate upper half of `pc + 4` and write it back to register `rd` in the Register File.

Branch Instructions

`beq rs1, rs2, label=imm`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Calculate jump target address (`pc` + `imm`) . Trigger a branch in the IF stage. Read register `rs2` from the Register File, buffer its first half for EXE stage and store its second half.	Buffer the stored second half of register `rs2` to EXE stage. Read register `rs1` from the Register File, forward its first half and buffer its second half for EXE stage.	
EXE		Compare first halves. If compare is decisive, signal IF stage the result.	Compare second halves. Signal IF stage the result.

Half ordering per instruction:
- Lower half first: BEQ, BNE
- Upper half first: BLT, BLTU, BGE, BGEU

CSR Register Instructions

```
csrrw rd, csr, rs1
csrrc rd, csr, rs1
csrrs rd, csr, rs1
```

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Buffer the `csr` address to EXE stage.	Read register `rs1` from the Register File, forward its lower half and buffer its upper half for EXE stage.	
EXE		Read the lower half of the CSR in address `csr` and write it back to register `rd` in the Register File. `csrrw`: Write the lower half of register `rs1` to the lower half of the CSR in address `csr`. `csrrs/c`: Set/Clear the bits of the lower half of the CSR in address `csr` according to the lower half of register `rs1`.	(Same for the upper halves) Read the upper half of the CSR in address `csr` and write it back to register `rd` in the Register File. `csrrw`: Write the upper half of register `rs1` to the upper half of the CSR in address `csr`. `csrrs/c`: Set/Clear the bits of the upper half of the CSR in address `csr` according to the upper half of register `rs1`.

CSR Immediate Instructions

```
csrrwi rd, csr, imm
csrrci rd, csr, imm
csrrsi rd, csr, imm
```

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Buffer the `csr` address to EXE stage.	Forward the lower half of the Immediate and buffer its upper half for EXE stage.	
EXE		Read the lower half of the CSR in address `csr` and write it	(Same for the upper halves) Read the upper half of the CSR

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
		back to register `rd` in the Register File. `csrrwi`: Write the lower half of the Immediate to the lower half of the CSR in address `csr`. `csrrs/ci`: Set/Clear the bits of the lower half of the CSR in address `csr` according to the lower half of the Immediate.	in address `csr` and write it back to register `rd` in the Register File. `csrrwi`: Write the upper half of the Immediate to the lower half of the CSR in address `csr`. `csrrs/ci`: Set/Clear the bits of the upper half of the CSR in address `csr` according to the upper half of the Immediate.

Accelerator Instructions

`sync, ld_key, st_enc, st_dec`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	`sync`: Stall until the accelerator is done. When it is done, write its output to registers `x28-x31`.		
EXE		`ld_key`: Load data in registers `x28-x31` as an encryption/decryption key. `st_enc/dec`: Start the encryption/decryption of data in registers `x28-x31`.	

Other Instructions

AUIPC

`auipc rd, imm`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Calculate `pc + imm`. Buffer the lower half of the result to EXE stage.	Buffer the upper half of the result to EXE stage.	
EXE		Write back the received half to register `rd` in the Register File.	Write back the received half to register `rd` in the Register File.

Load Immediate

`li rd, imm`
`lui rd, imm`

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Buffer the lower half of the Immediate to EXE stage.	Buffer the upper half of the Immediate to EXE stage.	
EXE		Write back the received half to register `rd` in the Register File.	Write back the received half to register `rd` in the Register File.

mret

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Forward the CSR `mepc` data to IF stage and signal jump.		

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
EXE		Restore the CSR `mstatus.mie` bit to its state prior to the interrupt.	

wfi

Cycle and execution step: Stage:	Cycle 3 - ID1	Cycle 4 - ID2, EXE1	Cycle 5 - EXE2
ID	Stall until an interrupt is triggered.		
EXE			

Theory Of Operation

Boot and Reset Sequence

Upon a hard reset (assertion of the active-low `rst_n` signal), the processor halts all execution and clears the internal pipeline stages.

1. The `mstatus` and `mie` CSRs are initialized to their reset states, masking all interrupts.
2. The Program Counter (PC) is synchronously loaded with the `INIT_PC` parameter (Default: 0x00000000).
3. On the next clock cycle following the de-assertion of `rst_n`, the Fetch Unit initiates an APB read transaction to the `INIT_PC` address to retrieve the first instruction.

Memory and CSR Map

The processor uses a modified Harvard architecture at the bus level, but software interacts with a unified 32-bit address space. Memory-mapped I/O (MMIO) and standard data memory are accessed via standard RISC-V load/store instructions over the Data APB interface.

Custom Control and Status Registers (CSRs)

In addition to the standard RISC-V Machine-mode CSRs (such as `mstatus`, `mtvec`, `mepc`, etc.), the core exposes custom hardware status registers to the software:

CSR Address	Name	Access	Description
0xFC0	AES_STATUS	Read-Only	Bit [0] indicates the accelerator readiness. 1 = Ready/Idle, 0 = Busy. Used for zero-overhead software polling.

Exception and Interrupt Handling

When the processor accepts an interrupt, the hardware automatically saves the current Program Counter (PC) into the mepc CSR, saves the interrupt enable state, globally disables further interrupts (mstatus.mie = 0), and jumps execution to the address stored in the mtvec (Machine Trap-Vector Base-Address) CSR.

Writing the Software Trap Handler

To properly resume execution after an asynchronous event, the software developer must implement a trap handler routine at the mtvec base address. The software routine is responsible for the following sequence:

1. **Context Saving:** The handler must immediately save the current state of the General Purpose Registers to the stack to avoid corrupting the interrupted program.
2. **Determine the Cause:** The software must read the mcause CSR to identify the source of the trap. RVEEAC utilizes the following specific exception codes:
    - **mcause = 11 (Machine External Interrupt):** Triggered by the external interrupt_req_ext_i pin. The software must communicate with the external interrupt controller over the APB bus to clear the source.
    - **mcause = 16 (Accelerator Interrupt):** Triggered by the internal AES accelerator finishing an operation. The software can safely retrieve the ciphertext from registers x28-x31.
3. **Clear the Interrupt:** For external interrupts, the software must acknowledge and clear the trigger at the source device. (Note: The internal AES interrupt clears automatically upon reading).
4. **Context Restoration:** The handler must pop the previously saved General Purpose Registers from the stack.
5. **Return from Trap:** The handler must conclude with the mret (Machine Return) instruction. The hardware will automatically restore the interrupt enable state (mstatus.mie) and jump back to the instruction address stored in mepc.

Enabling Interrupts

By default, the core boots with interrupts globally masked. To allow the processor to accept interrupts, the software must explicitly write a 1 to the mie (Machine Interrupt Enable) bit within the mstatus CSR.

Accelerator Software Interface

To facilitate high-performance AES operations, the processor implements a set of custom machine instructions. These instructions interact directly with the dedicated interface registers (`x28-x31`) and the accelerator's internal buffers. The interface uses a hardware back-pressure mechanism, meaning the processor pipeline will automatically stall if an instruction is issued while the accelerator is busy, eliminating the need for software polling.

Data Flow Model

The accelerator acts as a "Swap" engine coupled to registers `x28-x31`:

- **Input:** Issuing an encryption command copies the data from `x28-x31` into the accelerator's internal input buffer.
- **Output:** Issuing a synchronization command waits for completion and then copies the results from the accelerator's output buffer into `x28-x31`.

Synchronization Model: "Stall-on-Busy"

The accelerator interface is tightly coupled to the processor pipeline. The software does not need to poll a "Busy" bit:

- **Idle:** If the accelerator is idle, instructions execute immediately and the processor continues to the next cycle.
- **Busy:** If the accelerator is busy, issuing any AES instruction will automatically the processor pipeline. The pipeline resumes execution exactly one cycle after the accelerator finishes its current task.

Instruction Set

The following custom instruction control the accelerator.

Mnemonic	Description	Behavior
`AES_LOAD_KEY`	Key Load	Loads `x28-x31` to an internal key register. (Stalls if accelerator is busy)

Mnemonic	Description	Behavior
`AES_ENC`	Encrypt Block	Copies `x28-x31` to the internal buffer and starts encryption.
`AES_DEC`	Decrypt Block	Copies `x28-x31` to the internal buffer and starts decryption.
`AES_SYNC`	Sync & Retrieve	Stalls the processer until the accelerator is idle (if not already). Then copies the accelerator's internal buffer into `x28-x31`.
`AES_ENC_S`	Swap & Encrypt	1. Stalls until idle. 2. Swaps `x28-x31` with the accelerator internal buffer. 3. Starts encryption.
`AES_DEC_S`	Swap & Decrypt	Same as above but triggers decryption.

C-Code Example

To maximize encryption throughput, software can pipeline accelerator usage with core memory operations as shown below:

```
#define N 10

uint8_t key[16] = ...
uint8_t pts[N][16] = ...
uint8_t cts[N][16];

int main() {
     // 1. Load key
     LOAD_BLOCK_TO_REGS(key);
     AES_LOAD_KEY();

     // 2. Start first block
     LOAD_BLOCK_TO_REGS(pts[0]);
     AES_ENC(); // Copies x28-x31 to accel and starts. Regs are
now free.

     // 3. Pipeline loop
     for (int i = 1; i < N; i++) {
          // Load NEXT plaintext while accelerator works on
CURRENT block.
          LOAD_BLOCK_TO_REGS(pts[i]);

          // Swap: Wait for accel to finish, grab Ciphertext(i -
1), give Plaintext(i)
          AES_ENC_S();
```

```
        // Store the results we just retrieved
        STORE_BLOCK_FROM_REGS(cts[i-1]);
    }

    // 4. Drain the pipe (Retrieve last block)
    AES_SYNC(); // Wait for the last block and retrieve it.
    STORE_BLOCK_FROM_REGS(cts[N-1]);
```
``

- **Design Note:** As the instruction `AES_ENC_S` swaps the contents of the accelerator output and registers `x28-x31` atomically, it allows the processor to fetch the next data block into those registers *while* the accelerator is processing the previous block.

Execution Timing and Synchronization

To maximize encryption throughput, software developers must understand the data movement constraints and the timing overlap between the core and the accelerator.

Data Transfer (The 4-Word Constraint)

The AES-128 algorithm operates on 128-bit blocks. Because the RISC-V core utilizes a 32-bit architecture, the shared interface spans four distinct General Purpose Registers (x28, x29, x30, x31). Therefore, transferring a single AES block requires exactly four 32-bit LOAD instructions to populate the plaintext and four 32-bit STORE instructions to extract the ciphertext. Due to the core's 2-cycle APB memory interface, a full block transfer takes a minimum of 8 execution cycles to load, and 8 execution cycles to store.

Parametric Latency Balancing

The latency of the hardware accelerator is not fixed; it is defined by the SBOX_PAR_ROUND parameter. If instantiated for maximum throughput (16 parallel S-Boxes), the accelerator completes an encryption in roughly 11 cycles.

By utilizing the "Swap-on-Start" mechanism (AES_ENC_S), the software can execute the 8-cycle store and 8-cycle load sequence *while* the accelerator processes the previous block. This allows the hardware latency to be completely hidden behind the APB memory delays.

Supported Synchronization Methods

Depending on the hardware parametrization and the software environment, the system supports three distinct synchronization models to manage the overlap between the core and the accelerator:

1. **Hardware "Stall-on-Busy" (Implicit Synchronization):** If the software issues an AES_* instruction while the accelerator is still processing, the decode stage automatically stalls the pipeline. Execution resumes the exact cycle the accelerator finishes. This is the optimal method for tight, high-throughput polling loops where the AES latency closely matches the load/store overhead.
2. **Software Polling (CSR 0xFC0):** The software can read the custom AES_STATUS CSR. If bit 0 is 1, the accelerator is ready. This allows the core to perform other tasks and check the status without risking a pipeline stall.
3. **Asynchronous Interrupts:** Upon completion, the accelerator asserts an internal signal to the interrupt_sbm module, triggering a trap with mcause = 16. This method is highly recommended if the hardware is configured for minimum area (SBOX_PAR_ROUND = 1), which significantly increases encryption latency, allowing the CPU to execute other threads or enter a low-power wait-for-interrupt (WFI) state.

Optimal Pipelined Execution Timeline (Swap-on-Start)

The following table illustrates the parallel execution of the core and a high-throughput accelerator during an optimized AES_ENC_S loop.

Phase	**Core Execution (Memory & Pipeline)**	**Accelerator Execution (Background)**
Swap	AES_ENC_S (Swaps x28-x31 with AES buffers)	Latches new Plaintext, outputs old Ciphertext
Store CT	sw x28, 0(sp) *(2 APB cycles)*	Processing Round 1 & 2
	sw x29, 4(sp) *(2 APB cycles)*	Processing Round 3 & 4
	sw x30, 8(sp) *(2 APB cycles)*	Processing Round 5 & 6

Phase	Core Execution (Memory & Pipeline)	Accelerator Execution (Background)
	sw x31, 12(sp) *(2 APB cycles)*	Processing Round 7 & 8
Load PT	lw x28, 16(sp) *(2 APB cycles)*	Processing Round 9 & 10
	lw x29, 20(sp) *(2 APB cycles)*	**Done** (Enters IDLE state)
	lw x30, 24(sp) *(2 APB cycles)*	Idle
	lw x31, 28(sp) *(2 APB cycles)*	Idle
Loop	AES_ENC_S (Swaps data for next block)	Latches new Plaintext, outputs old Ciphertext

Note: In this optimized configuration, the accelerator finishes before the core completes the memory transfers, meaning the next AES_ENC_S instruction executes instantly without triggering a hardware stall.

Area-Optimized Execution Timeline (SBOX_PAR_ROUND = 1)

If the system integrator prioritizes minimum silicon area and lower peak power, the accelerator can be parameterized to use fewer S-Boxes. In the absolute minimal configuration, a single encryption takes approximately 160 clock cycles.

In this scenario, the software finishes its memory transfers long before the accelerator completes its task. To maximize system efficiency, the software should utilize the Asynchronous Interrupt method. This completely frees the core to either enter a low-power wait state or switch contexts to execute other software tasks while the accelerator works in the background.

Phase	Core Execution (Memory & Pipeline)	Accelerator Execution (Background)
Swap	AES_ENC_S (Swaps data & starts Accel)	Latches new Plaintext, outputs old Ciphertext
Store CT	4x sw instructions *(~8 APB cycles)*	Processing Round 1 *(Cycles 1-16)*
Load PT	4x lw instructions *(~8 APB cycles)*	Processing Round 2 *(Cycles 17-32)*
Sleep / Multitask	WFI (Low Power) **OR** Execute other threads	Processing Rounds 3 through 9 *(Cycles 33 - 144)*
	(Sleeping or Multitasking)	Processing Round 10 *(Cycles 145 - 160)*
Interrupt	Wake up / Context Switch on mcause = 16	**Done** (Triggers Interrupt)
Loop	AES_ENC_S (Swaps data for next block)	Latches new Plaintext, outputs old Ciphertext

Note: By utilizing the Asynchronous Interrupt method in the area-optimized configuration, the processor core is freed for over 140 cycles per block. The system can either safely power down its execution units (via WFI) to drastically reduce energy consumption, or utilize those cycles to handle other peripherals and software threads, achieving true hardware-software concurrency.

Verification

Verification Methodology

The verification strategy employed a hybrid bottom-up and top-down approach, ensuring both the correctness of critical arithmetic units and the architectural compliance of the full system.

1. Module-Level Fuzzing (SystemVerilog): Critical combinational logic, specifically the Arithmetic Logic Unit (ALU), was verified using a constrained-random fuzzer. A SystemVerilog testbench (tb_alu_sbm.sv) generated random inputs for all supported operations (ADD, SUB, Logic, Shifts, Comparisons)1. The results were checked against a behavioral model (serialize task) to ensure bit-perfect accuracy across 200,000 iterations.

2. Directed Pipeline Verification (SystemVerilog): The core pipeline was validated using a directed testbench (tb_core.sv) that manually injected instruction sequences. This targeted specific micro-architectural corner cases including:

   - **Immediate generation and Register Write-back:** Verifying correct sign-extension and destination routing.
   - **Data Hazards:** Testing forwarding paths and stall logic by executing dependent instructions (Read-After-Write) in close succession.

- **Control Flow:** Verifying conditional branches (BEQ, BNE, BLT) and unconditional jumps (JAL, JALR), ensuring the PC updates correctly and the pipeline flushes on mispredictions.

3. System-Level Software Verification (C/C++): The fully integrated core was verified by running compiled C programs (tb_core_c_program.sv). To ensure robustness against realistic bus latencies, the memory models were configured to inject random wait states (via the POSSIBLE_WAITS parameter). This verified the core's ability to correctly stall and resume execution under variable instruction fetch and data access delays, alongside validating standard control flow and I/O.

Verification Results

The design passed all directed and random verification tiers. The following table summarizes the key test suites executed:

Test Suite	Description	Verification Method	Result
ALU Fuzzer	Random input vectors for Integer and Logic ops.	SV Assertions vs Model (200k iter)	**PASS**
Pipeline Hazards	Directed RAW dependencies and Load-Use stalls.	SV Assertions on Register State	**PASS**
Control Logic	Branch prediction validation (Taken/Not Taken).	SV Assertions on PC Value	**PASS**
Standard Runtime	Stack frames, Global variables, `malloc`, `printf`.	C execution (`simple.c`)	**PASS**
CSR Operations	Atomic Read/Write/Set/Clear on `mscratch`.	C Assertions (`test_csr.c`)	**PASS**
Interrupts	Enable/Disable logic and Trap handling.	C execution (`test_interrupt.c`)	**PASS**
AES Hardware	Encryption/Decryption correctness.	SW vs HW output match (`test_accel.c`)	**PASS**

AES – Advanced Encryption Standard:

The Advanced Encryption Standard (AES) is a widely adopted symmetric block-cipher algorithm. It operates on fixed 128-bit blocks of data and supports three key lengths—128, 192, or 256 bits—which correspond to 10, 12, or 14 rounds of processing, respectively. Each round applies a series of transformations—SubBytes (a non-linear byte substitution using an S-box), ShiftRows (a transposition step that cyclically shifts each row of the state), MixColumns (a linear mixing operation over each column), and AddRoundKey (an XOR with a portion of the expanded key)—to thoroughly diffuse and obscure the plaintext. The advantages of AES is a combination of strong security, high performance in both hardware and software, and straightforward, regular structure, making it the backbone of modern secure communications, disk encryption, and countless cryptographic protocols.

GF (2^8)– Galois Field:

Each byte in the encryption process is treated as an element in the finite field GF (2^8). The byte is observed in a polynomial representation, such as:

$$b = b_7x^7 + b_6x^6 + \cdots + b_0x^0 \quad , b_i \in \{0,1\}$$

Over this field 2 operation are relevant to the encryption process, addition and multiplication. Addition is a bitwise XOR. Multiplication is multiplying 2 bytes then using the modulo operation using the AES irreducible polynomial:

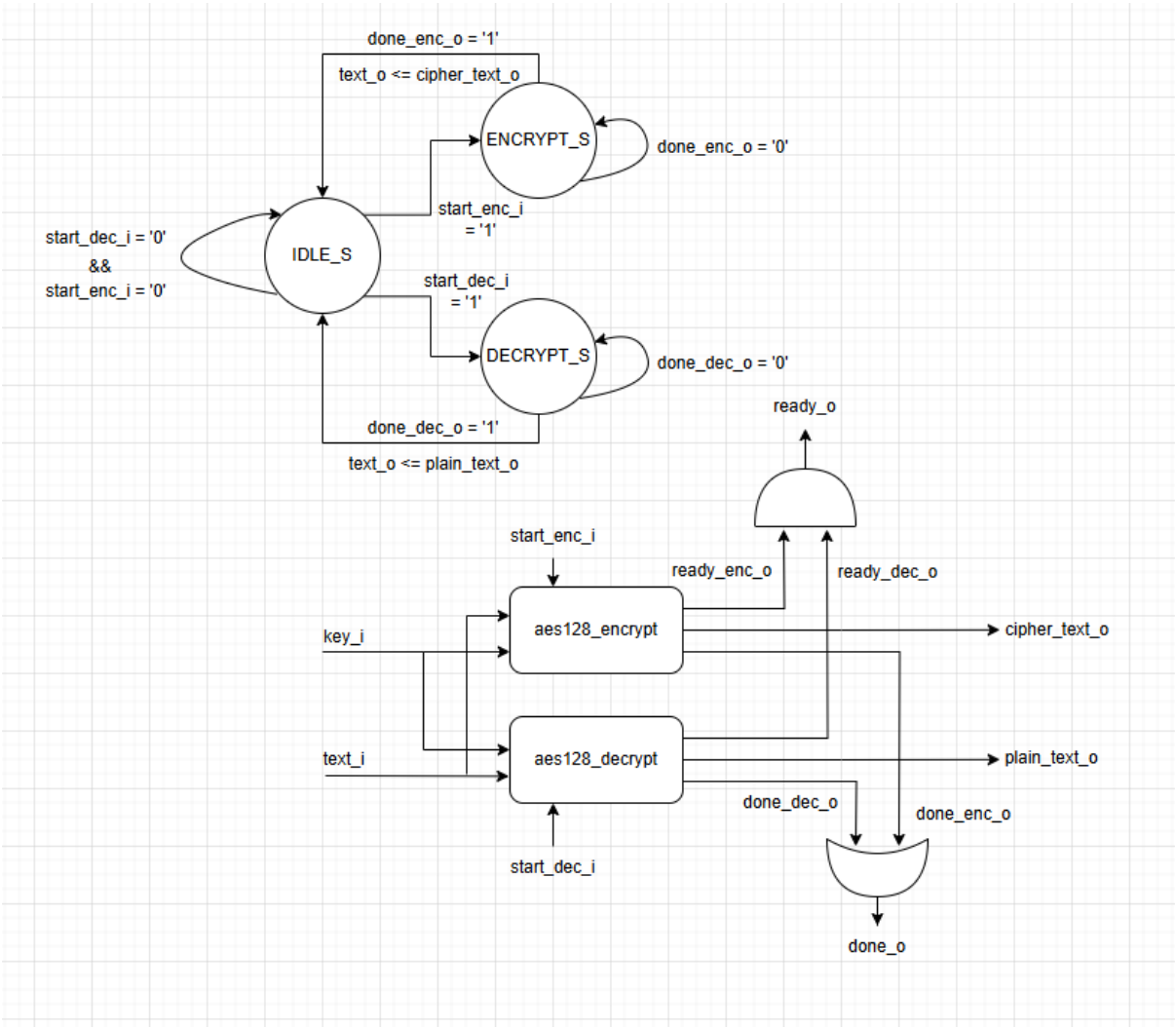
$$m(x) = x^8 + x\text{^} + x^3 + x + 1 = 0\text{x}1\text{B}$$

Practically, Multiplication is implemented in hardware using a "Multiply by 2" base block, which involves hardware cheap operation: shift and XOR. AES uses this finite field to gain diffusion and non-linearity.

Modules

aes128_core

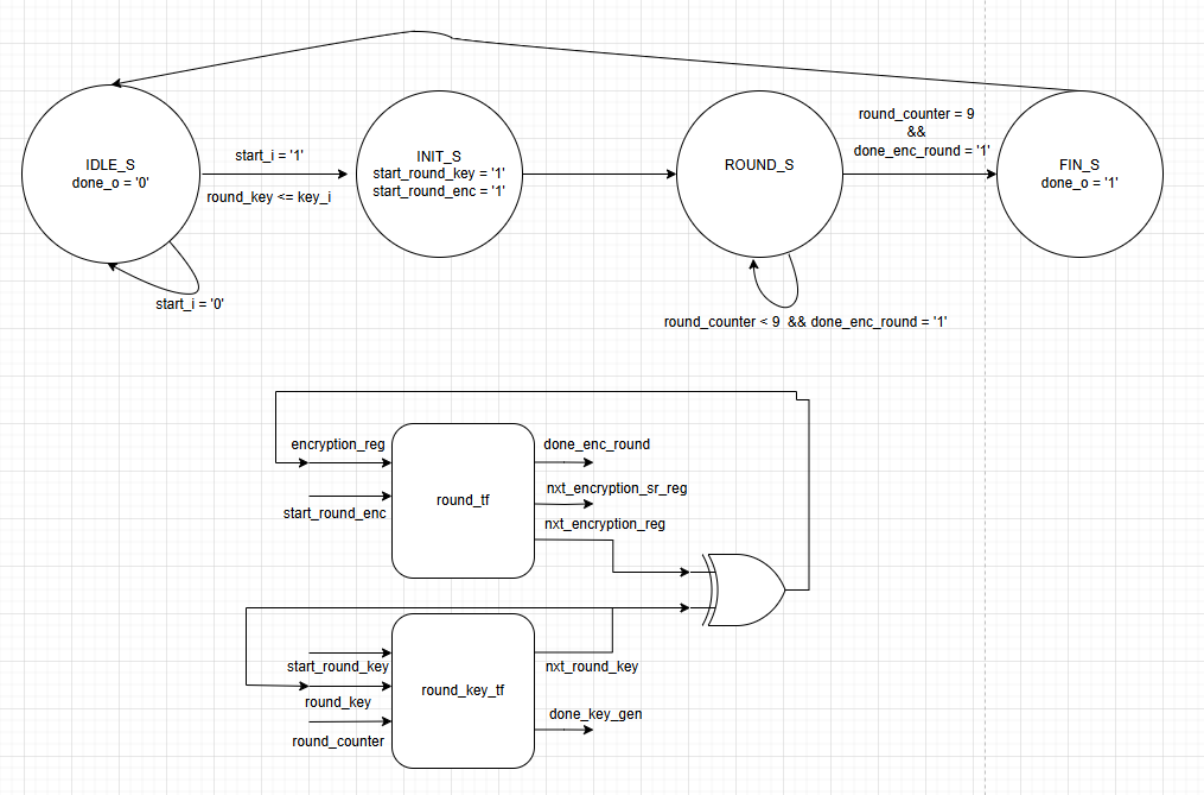
The aes128_core module serves as the top-level controller for AES-128 cryptographic operations, integrating both encryption and decryption capabilities into a single cohesive unit. It receives a 128-bit input (text_i) along with a 128-bit key (key_i) and delegates processing to either the aes128_encrypt or aes128_decrypt submodule, depending on which start signal is asserted. Internally, the module uses a simple FSM to toggle between idle, encryption, and decryption states. Once an operation completes, the corresponding output (enc_text_o or dec_text_o) is routed to text_o, and a done_o pulse is generated. The ready_o signal ensures that no operation starts unless both encryption and decryption units are idle. This modular architecture allows for clean integration into systems requiring runtime AES mode selection, while maintaining clear separation of encryption and decryption logic under a unified interface.



aes128_encrypt

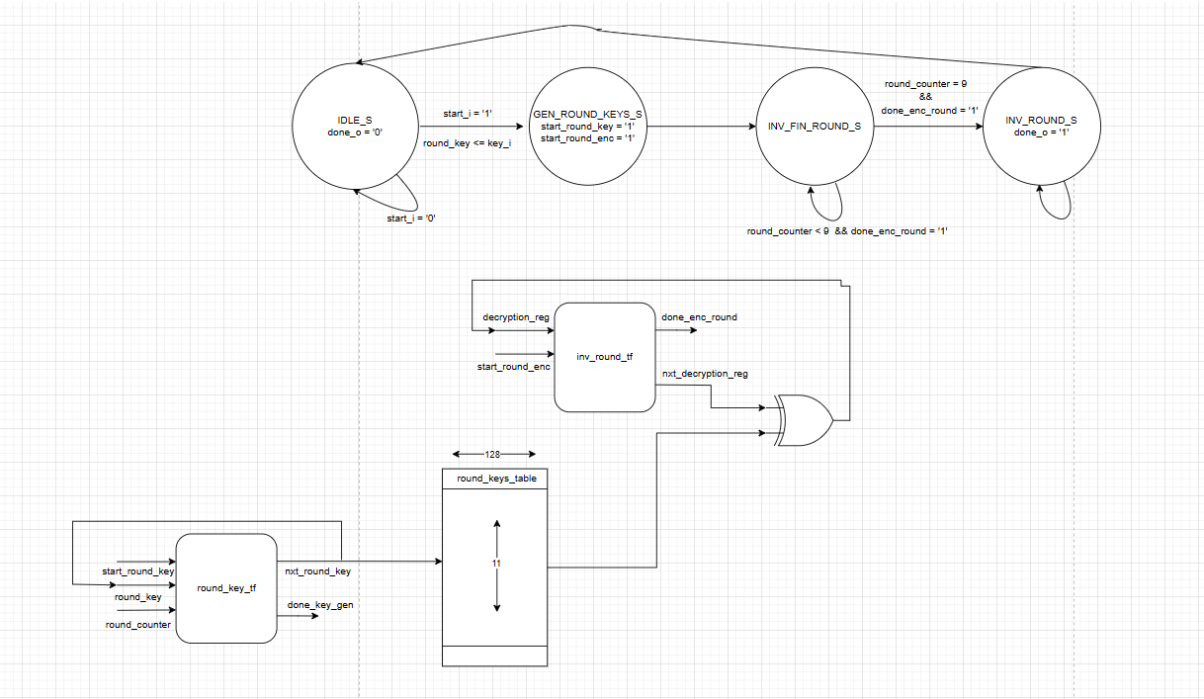
The aes128_encrypt module orchestrates the full AES-128 encryption process, handling the sequencing and control of each encryption round. It accepts a 128-bit plaintext and a 128-bit secret key and outputs the corresponding 128-bit ciphertext. Internally, the design is driven by a finite state machine (FSM) that transitions

through four states: IDLE, INIT, ROUND, and FIN, corresponding to the stages of AES encryption. Initially, the plaintext is XORed with the first-round key (AddRoundKey). For rounds 1 to 9, the module invokes two submodules: round_tf (handling SubBytes, ShiftRows, and MixColumns) and round_key_tf (generating the next round key). The final round omits the MixColumns step as per the AES standard. Data and key registers are updated synchronously, with precise control of start and done signals to manage pipeline timing. This module ensures full compliance with the AES-128 specification while maintaining clean control logic and extensibility for integration into higher-level systems.



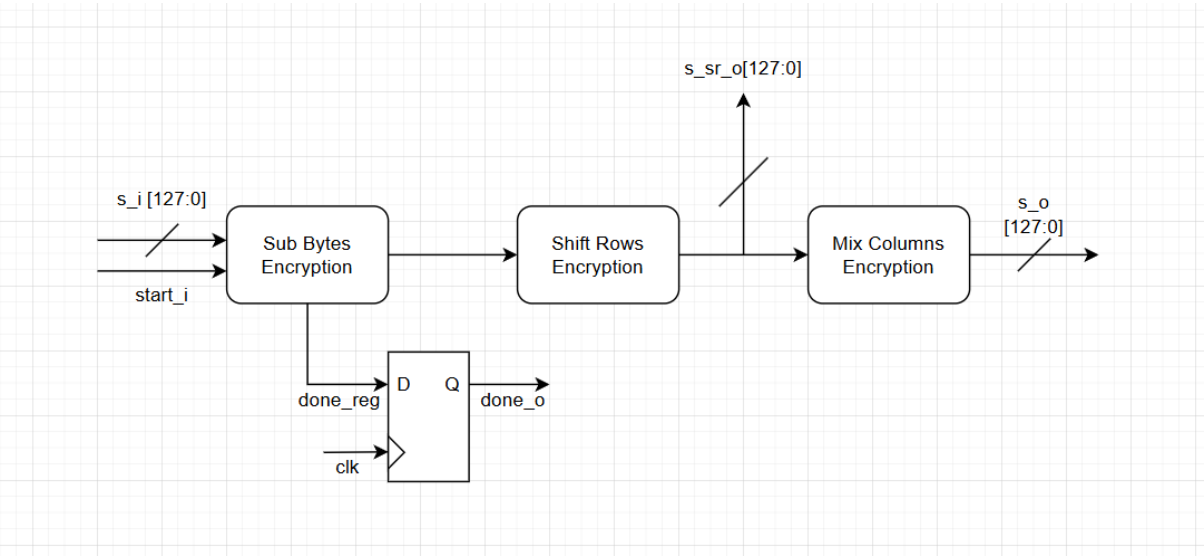
Aes128_decrypt

The aes128_decrypt module implements the inverse AES-128 algorithm to recover plaintext from a 128-bit ciphertext using the original encryption key. It begins by generating all 11 round keys (from round 0 to 10) using the same key schedule logic as the encryption path. These keys are stored in a local register file, allowing direct access in reverse order for decryption. The decryption process then follows the AES standard in reverse: the final round is handled first (applying InvShiftRows, InvSubBytes, and AddRoundKey, but bypassing InvMixColumns), followed by rounds 9 to 1 which include the full inverse transformation including InvMixColumns. The last round (r=0) completes the inverse operation, yielding the original plaintext. Control is handled via a finite state machine with states to manage key expansion, round execution, and result delivery. Key design choices such as precomputing round keys and selectively bypassing MixColumns ensure accurate and efficient hardware decryption flow, fully compliant with the AES specification.



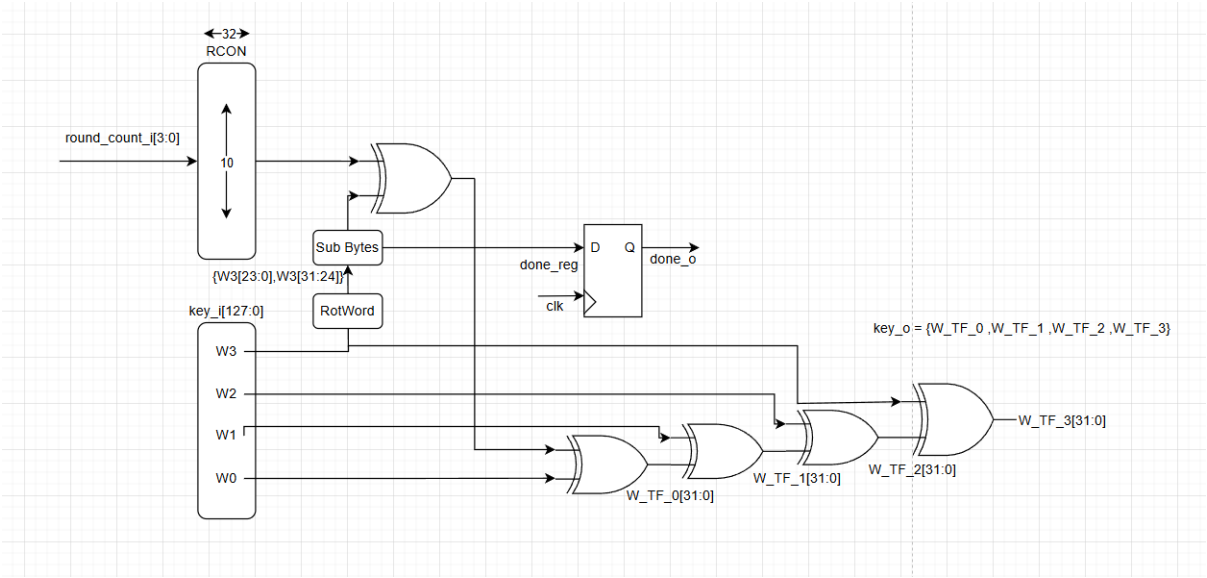
Round_tf

The round_tf module performs a single standard AES-128 encryption round, applying the three core transformations: SubBytes, ShiftRows, and MixColumns. It receives a 128-bit input state (s_i) and produces two outputs: the intermediate state after the ShiftRows operation (s_sr_o) and the final state after MixColumns (s_o). Internally, the module first uses the sub_bytes unit (parameterized for encryption) to substitute each byte via the AES S-box. The result is passed through the shift_rows module, which cyclically shifts the rows of the state matrix to the left. Finally, the mix_columns transformation mixes each column using Galois Field arithmetic. A done_o signal is pipelined from the S-box unit to signal completion, ensuring alignment with clock timing. This modular structure facilitates clean round-level reuse in both encryption and decryption contexts.



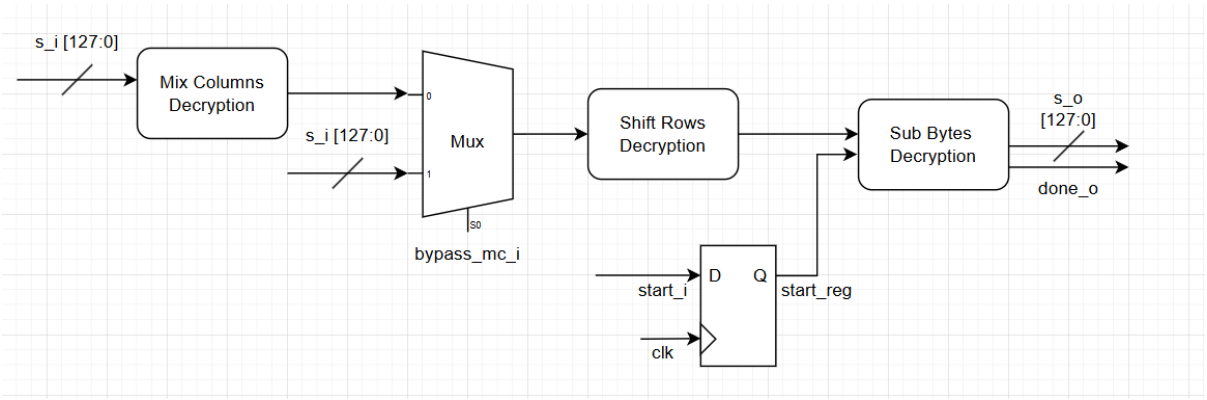
Round_key_tf

The round_key_tf module performs the key schedule operation for AES-128, generating the next round key from the current one during encryption or decryption. It accepts a 128-bit input key (key_i) and a 4-bit round index (round_count_i), producing the corresponding next 128-bit round key (key_o). The module follows the AES key expansion algorithm: it rotates the last word (w[3]), applies an S-box substitution to the rotated word, and then XORs it with the first word and the round constant (RCON). The remaining words are computed as chained XORs of previous results. To ensure synchronization with surrounding logic, the S-box substitution is handled in a pipelined manner with a done_o flag signaling completion. It forms a key component of both encryption and decryption flows by enabling dynamic round key generation on a per-cycle basis.



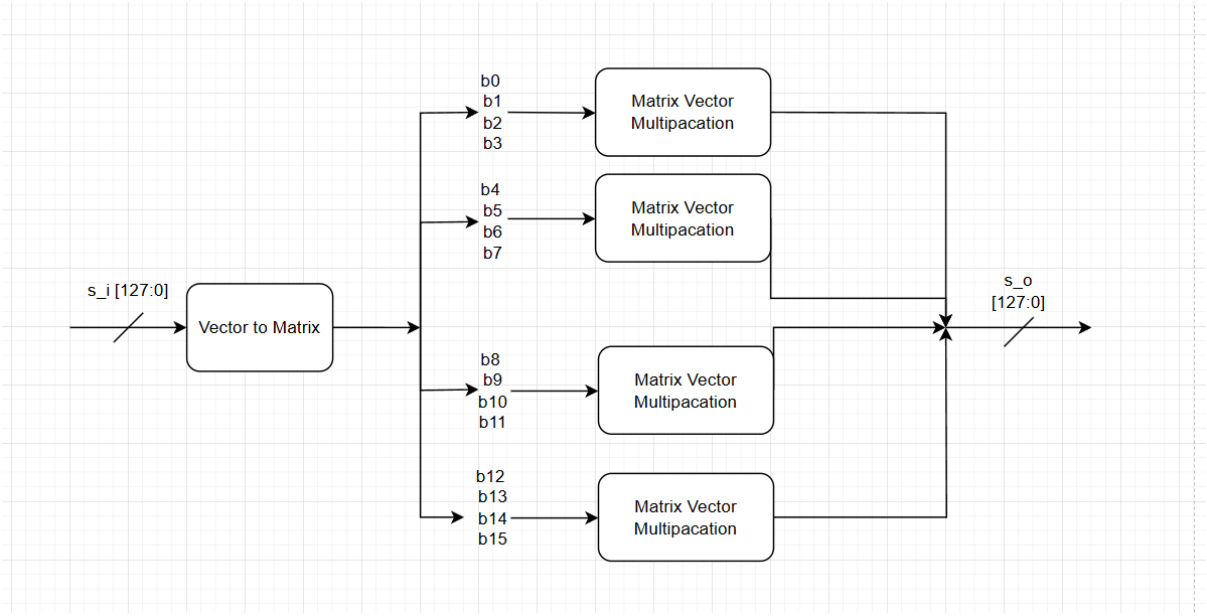
Inv_round_tf

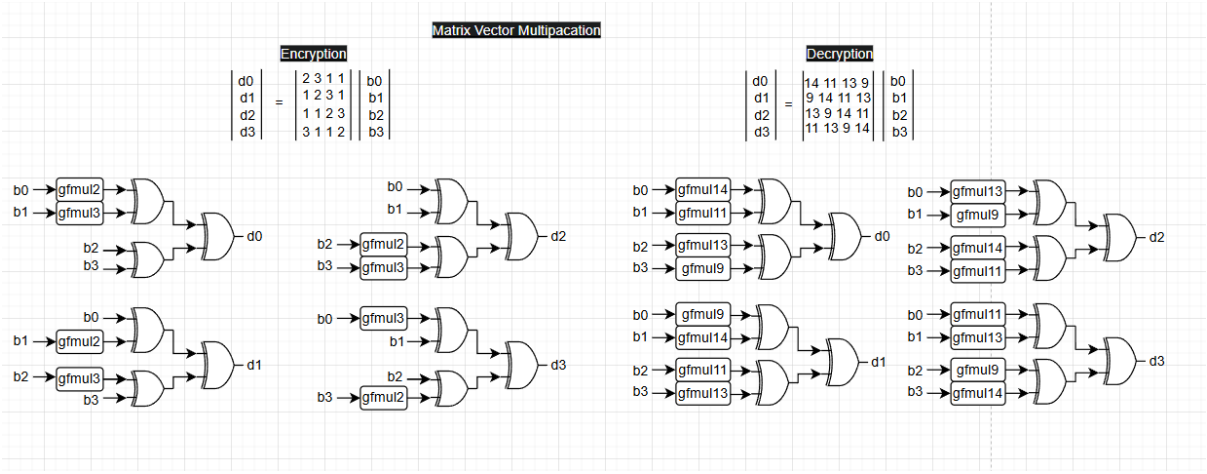
The inv_round_tf module implements one full decryption round of the AES-128 algorithm, applying the inverse transformations of MixColumns, ShiftRows, and SubBytes. It accepts a 128-bit state (s_i) and produces the transformed output (s_o) along with a done_o flag to signal completion. A control input bypass_mc_i enables skipping the inverse MixColumns step, which is required during the final AES round. If bypass_mc_i is asserted, the state bypasses the inverse MixColumns unit and proceeds directly to inverse ShiftRows. The shift_rows and sub_bytes components are both configured for decryption (OP=0), ensuring correct application of inverse permutations and substitutions. The S-box operation is pipelined using a start_reg to align timing with start_i. This modular structure is critical for the decryption flow in AES-128, where each round applies the inverse transformations in reverse order compared to encryption.



Mix_columns

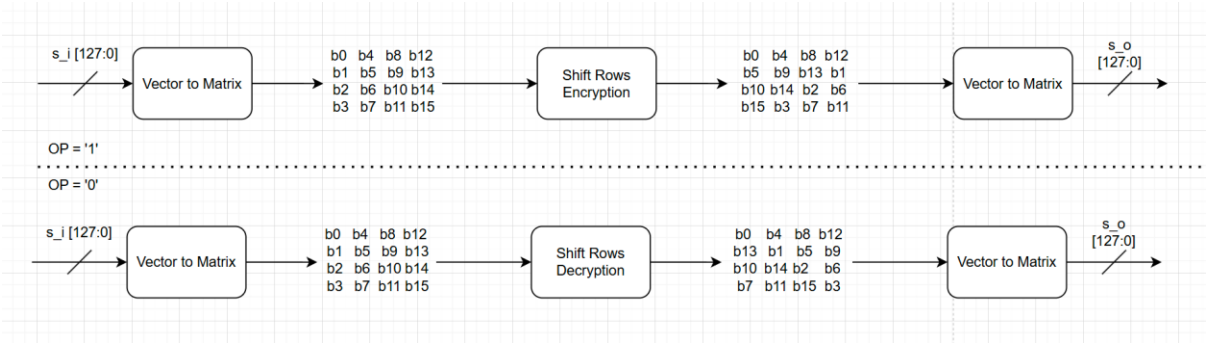
The mix_columns module implements the MixColumns operation of AES, responsible for inter-column diffusion in the 4×4 byte state matrix. It is parameterized by OP, where OP=1 enables encryption mode and OP=0 enables decryption mode. Internally, the 128-bit input state (s_i) is first unpacked into an array of 16 bytes. Each of the four 4-byte columns is then processed independently using Galois Field multiplications. In encryption mode, each output byte is calculated using the fixed matrix {2,3,1,1}, while in decryption mode, it uses the inverse matrix {14,11,13,9}. The appropriate finite field multiplications (gfmul2, gfmul3, gfmul9, etc.) are imported from an external function package. The resulting bytes are repacked into the 128-bit output state (s_o). This module ensures accurate column mixing as required by the AES specification and supports both forward and inverse transformations through a single unified implementation.





Shift_rows

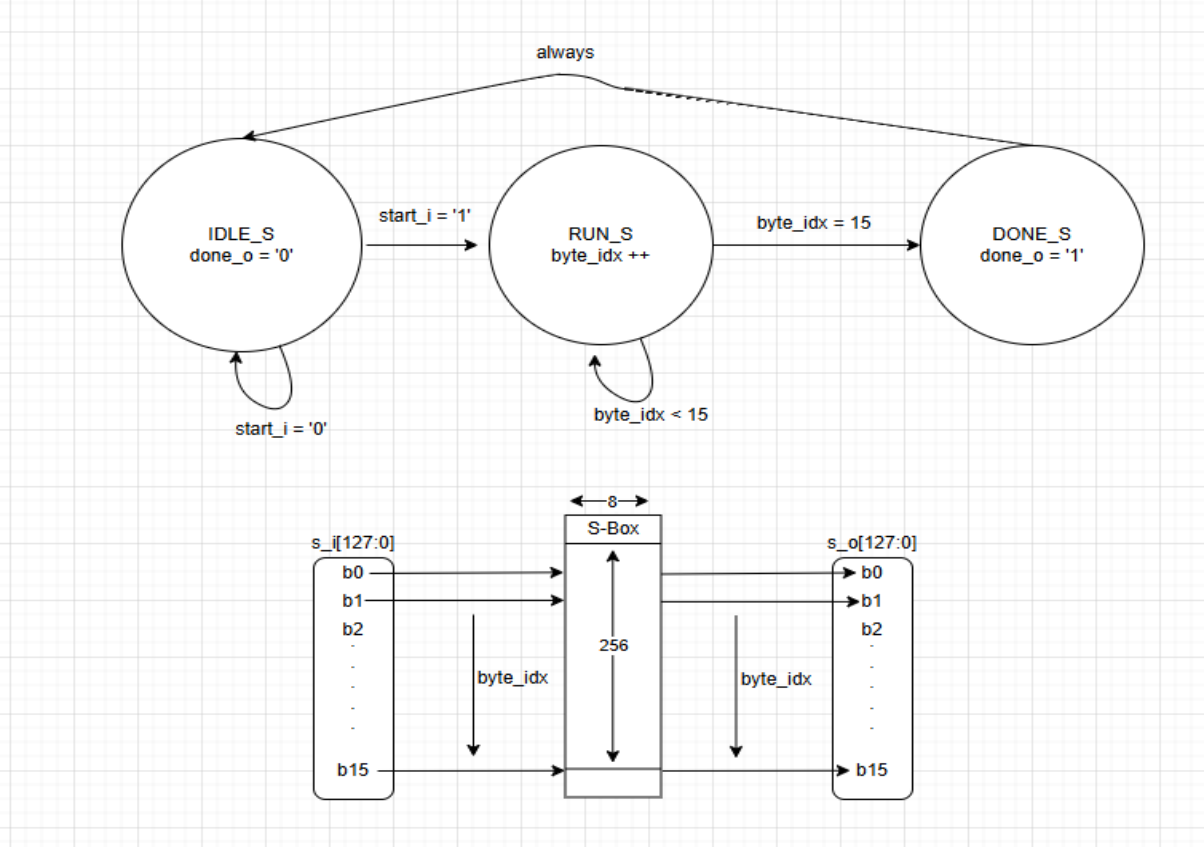
The shift_rows module performs the row-wise permutation step of AES, which cyclically shifts the bytes within each row of the 4×4 state matrix. This operation enhances diffusion by disrupting the byte alignment across columns. The module is parameterized by OP, where OP=1 applies the standard **left shift** used during encryption, and OP=0 applies the **right shift** used in decryption. The 128-bit input (s_i) is first unpacked into 16 individual bytes, preserving their matrix order. Depending on the mode, the module rearranges these bytes according to the AES specification: encryption shifts rows by offsets {0, 1, 2, 3}, while decryption shifts them in reverse {0, -1, -2, -3}. The transformed bytes are then packed back into the output state (s_o). This module is purely combinational and efficiently implements a critical structural transformation in AES processing.



Sub_bytes

The sub_bytes module implements the SubBytes transformation of AES, a non-linear substitution that replaces each byte of the input state using a substitution box (S-box). Parameterized by WIDTH (typically 128 bits) and

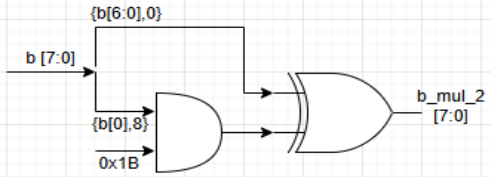
OP (operation mode), it supports both encryption (OP=1, using the forward S-box) and decryption (OP=0, using the inverse S-box). The S-box values are loaded from external memory files (sbox_table.mem or inv_sbox_table.mem) into a 256-entry ROM. Internally, the module uses a simple FSM with states IDLE, RUN, and DONE, processing one byte per clock cycle in the RUN state. Each byte of the input (s_i) is substituted and stored into a temporary result register (temp_res), which is then output as s_o once all bytes are processed. The done_o signal marks the end of substitution. This design prioritizes simplicity and timing compatibility with pipelined systems, while still allowing optional substitution with a combinational implementation if desired.



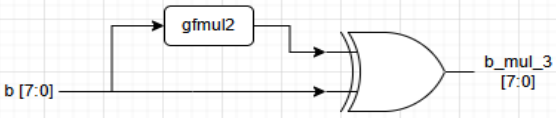
Aes_pack_functions

The hea_func_pack package defines a suite of utility functions for performing finite field ($GF(2^8)$) multiplications, which are fundamental to the AES MixColumns and InvMixColumns transformations. It provides optimized implementations for multiplying a byte by the constants used in AES: {2,3,4,8,9,11,13,14}. These functions exploit the properties of $GF(2^8)$ arithmetic, using bitwise shifts and conditional XORs with the AES irreducible polynomial (0x1B) to efficiently compute the results. For instance, gfmul2 performs a left shift with conditional reduction, while gfmul3 is a combination of gfmul2 and the original byte. Composite multipliers like gfmul14 chain together basic operations to reduce redundant logic. By centralizing these operations, the package promotes code reuse, modularity, and readability across all AES-related modules requiring Galois field arithmetic.

gfmul2



gfmul3

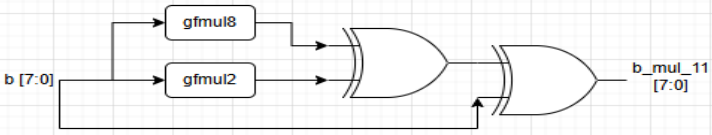


gfmul4

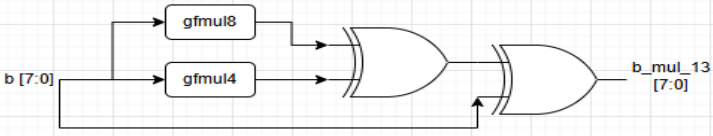
gfmul8

gfmul9

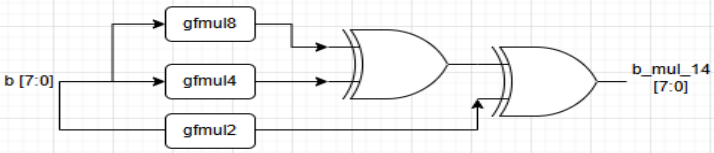
gfmul11



gfmul13

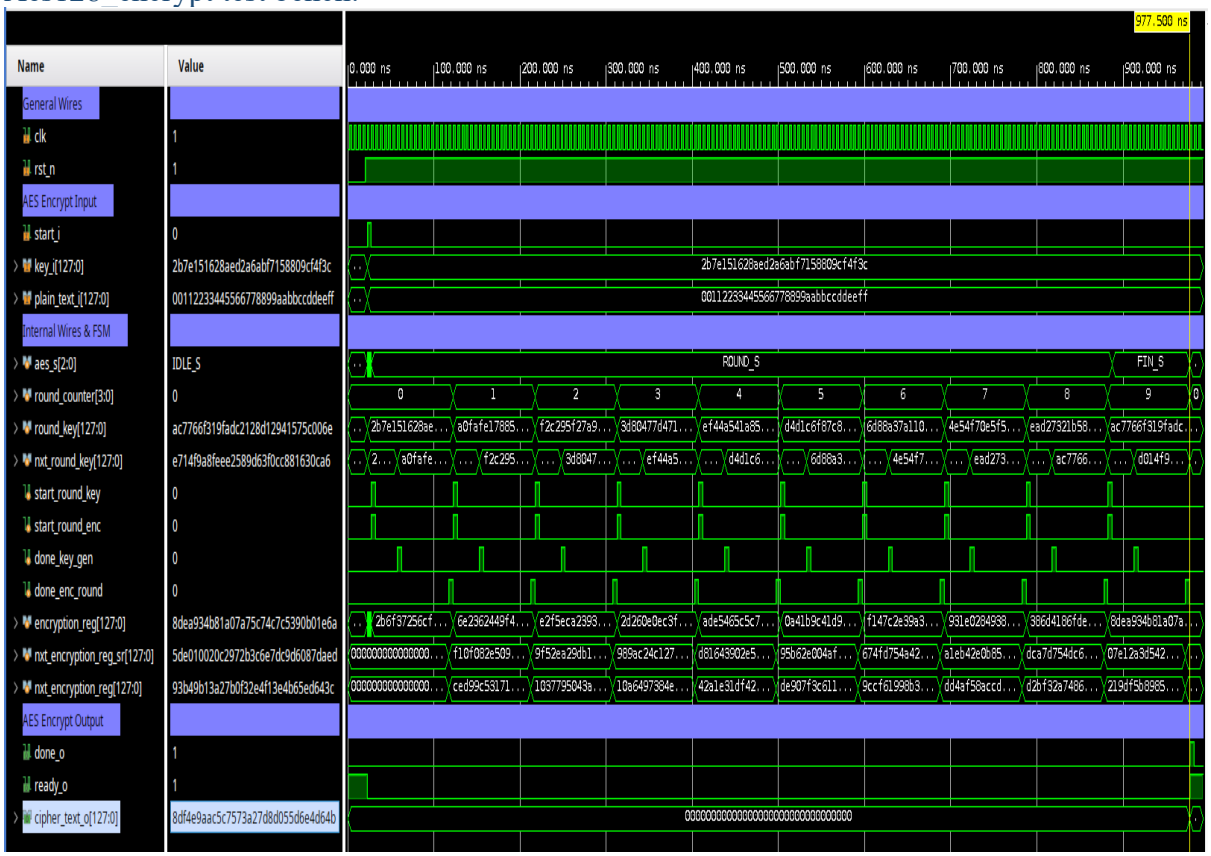


gfmul14



Simulation examples:

Aes128_encrypt test bench:



This waveform captures the full AES-128 encryption process as simulated in the testbench. The input plaintext (plain_text_i) is 00112233445566778899aabbccddeeff, and the key (key_i) is 2b7e151628aed2a6abf7158809cf4f3c, both consistent with the AES FIPS-197 test vector.

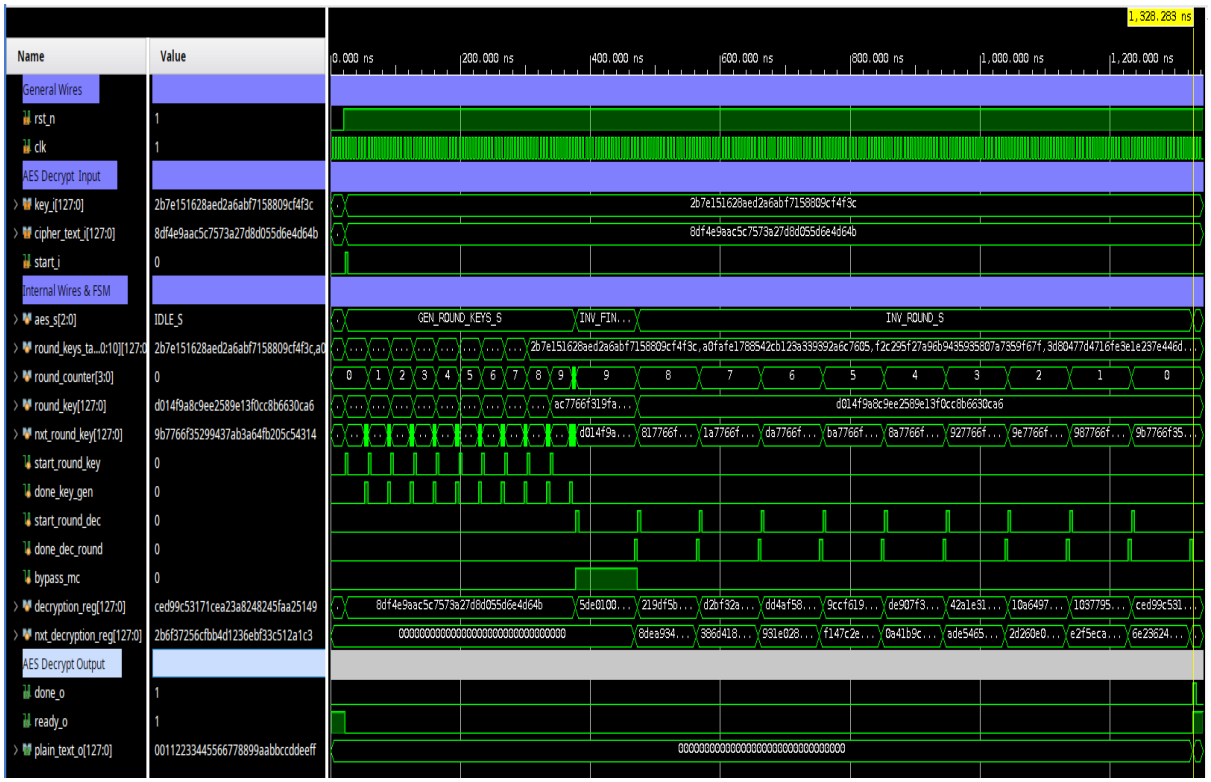
The simulation begins with a reset (rst_n = 0 → 1) and triggers encryption via start_i. The FSM (aes_s) transitions from IDLE_S to INIT_S, then into ROUND_S as it progresses through the 10 AES rounds. Each round involves a new round_key, generated and shown per cycle, and a new internal state held in encryption_reg.

- **round_counter** tracks the current AES round (0–9).
- **start_round_enc** and **start_round_key** pulse each time a new encryption or key schedule step starts.
- **done_enc_round** and **done_key_gen** indicate completion of round logic and key generation respectively.

During the FIN_S state, the final round executes (without MixColumns), and the ciphertext output (cipher_text_o) stabilizes at 8d4e9aac5c7573a27d8d055d6e4d064b, matching the expected AES result. The done_o and ready_o signals assert, indicating completion and readiness for the next operation.

This simulation confirms functional correctness of the AES encryption core, including key expansion, round processing, and output matching reference vectors.

Aes128_decrypt test bench:



This waveform captures the simulation of the AES-128 **decryption process**, verifying that the ciphertext is correctly decrypted back to the original plaintext. The ciphertext input (cipher_text_i) is 8df4e9aac5c7573a27d8d055d6e4d064b, and the key (key_i) is 2b7e151628aed2a6abf7158809cf4f3c, both matching the FIPS-197 test vector for AES-128.

The module begins in the IDLE_S state. Once start_i is asserted, the FSM enters GEN_ROUND_KEYS_S, where round keys are generated sequentially (0 through 10), stored in round_keys_table, and indexed by round_counter. Once key generation is complete, the FSM transitions to INV_FIN_ROUND_S for the final round (which skips InvMixColumns using bypass_mc = 1), followed by the remaining 9 inverse rounds in INV_ROUND_S.

- **decryption_reg** and **nxt_decryption_reg** show the internal state transformation over time.
- **start_round_dec** and **done_dec_round** control the execution of each decryption round.
- At the end of the process, the decrypted output (plain_text_o) matches the original input message: 00112233445566778899aabbccddeeff

The handshake signals done_o = 1 and ready_o = 1 indicate successful decryption and readiness for the next operation. This confirms that the AES decryption logic, including key schedule and inverse round transformations, is functioning correctly.

Summary and Conclusion

This project demonstrates how a compact RV32I processor can be extended with a dedicated AES accelerator while keeping the software interface simple. The main idea was to use a 16-bit split data path to reduce hardware complexity, area, and power, while still supporting the full 32-bit RV32I architecture. Through this design, we learned about the trade-offs between performance, pipeline control, hardware cost, and software usability.

Future work can focus on completing the physical implementation and improving the verification process. On the implementation side, the next steps are to synthesize the design, analyse timing, area, and power reports, and continue toward floor planning, placement, routing, and layout generation. On the verification side, the system should be tested with more random instruction streams, additional hazard and interrupt corner cases, and longer programs that combine memory access, CSR operations, control flow, and AES acceleration.